



Siemens
Industry
Online
Support

SIMATIC Source Documents

LADDER Format Description

A textual representation for LADDER graphical programming language

TIA Portal V20

SIEMENS

Legal information

Use of application examples

Application examples illustrate the solution of automation tasks through an interaction of several components in the form of text, graphics and/or software modules. The application examples are a free service by Siemens AG and/or a subsidiary of Siemens AG ("Siemens"). They are non-binding and make no claim to completeness or functionality regarding configuration and equipment. The application examples merely offer help with typical tasks; they do not constitute customer-specific solutions. **The application examples are not subject to standard tests and quality inspections of a chargeable product and may contain functional and performance defects or other faults and security vulnerabilities. You are responsible for the proper and safe operation of the products in accordance with all applicable regulations, including checking and customizing the application example for your system, and ensuring that only trained personnel use it in a way that prevents property damage or injury to persons. You are solely responsible for any productive use.**

Siemens grants you the non-exclusive, non-sublicensable and non-transferable right to have the application examples used by technically trained personnel. Any change to the application examples is your responsibility. Sharing the application examples with third parties or copying the application examples or excerpts thereof is permitted only in combination with your own products. Any further use of the application examples is explicitly not permitted and further rights are not granted. You are not allowed to use application examples in any other way, including, without limitation, for any direct or indirect training or enhancements of AI models.

Disclaimer of liability

Siemens shall not assume any liability, for any legal reason whatsoever, including, without limitation, liability for the usability, availability, completeness and freedom from defects of the application examples as well as for related information, configuration and performance data and any damage caused thereby. This shall not apply in cases of mandatory liability, for example under the German Product Liability Act, or in cases of intent, gross negligence, or culpable loss of life, bodily injury or damage to health, non-compliance with a guarantee, fraudulent non-disclosure of a defect, or culpable breach of material contractual obligations. Claims for damages arising from a breach of material contractual obligations shall however be limited to the foreseeable damage typical of the type of agreement, unless liability arises from intent or gross negligence or is based on loss of life, bodily injury or damage to health. The foregoing provisions do not imply any change in the burden of proof to your detriment. You shall indemnify Siemens against existing or future claims of third parties in this connection except where Siemens is mandatorily liable.

By using the application examples you acknowledge that Siemens cannot be held liable for any damage beyond the liability provisions described.

Other information

Siemens reserves the right to make changes to the application examples at any time without notice and to terminate your use of the application examples at any time. In case of discrepancies between the suggestions in the application examples and other Siemens publications such as catalogs, the content of the other documentation shall have precedence.

The Siemens terms of use (<https://www.siemens.com/global/en/general/terms-of-use.html>) shall also apply.

Cybersecurity information

Siemens provides products and solutions with industrial cybersecurity functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial cybersecurity concept. Siemens' products and solutions constitute one element of such a concept.

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place.

For additional information on industrial cybersecurity measures that may be implemented, please visit www.siemens.com/cybersecurity-industry.

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Cybersecurity RSS Feed under <https://www.siemens.com/cert>.

Table of contents

- 1. Introduction5**
 - 1.1. First Example..... 6
 - 1.2. IEC 61131-3 7
 - 1.3. STEP 7..... 8
 - 1.4. Goals 8
- 2. General Concepts10**
 - 2.1. Blocks 10
 - 2.2. Constants..... 10
 - 2.3. Interfaces..... 11
 - 2.4. Networks 11
- 3. Ladder Concepts.....12**
 - 3.1. Logic..... 12
 - 3.2. Gates 12
 - 3.3. Pins..... 14
 - 3.3.1. Rung-In, Rung-Out..... 15
 - 3.3.2. Operand..... 15
 - 3.3.3. Top, Bottom 16
 - 3.3.4. Instance 16
 - 3.4. Templates 17
 - 3.5. Parts 18
 - 3.6. Wires 19
 - 3.7. Rungs 21
 - 3.8. Summary 24
- 4. Instructions25**
 - 4.1. Contacts..... 25
 - 4.1.1. Contacts With One Operand 25
 - 4.1.2. Contacts With Two Operands 26
 - 4.1.3. Contacts Without Operands 27
 - 4.2. Coils 28
 - 4.2.1. Coils With One Operand..... 28
 - 4.2.2. Coils With Two Operands 29
 - 4.2.3. Coils Without Operands 30
 - 4.3. Negation..... 31
 - 4.4. ENO-Boxes 32
 - 4.4.1. ENO Calculation On and Off 33
 - 4.4.2. Implicit Converts 34

4.4.3.	Calls.....	35
4.5.	Q-Boxes	37
4.5.1.	Bistable Function Blocks.....	37
4.5.2.	Timer	39
4.5.3.	Counter	41
4.6.	Jumping.....	43
4.6.1.	Call	43
4.6.2.	Return.....	43
4.6.3.	Labels	44
4.6.4.	Jumps	45
4.6.5.	Only One Jump or Return	45
4.6.6.	Jump List.....	47
4.6.7.	Switch	48
4.7.	Calculating.....	50
5.	Export, Import.....	51
6.	Appendix	53
6.1.	Service and support	53
6.2.	Links and literature	54
6.3.	Change documentation	54

1. Introduction

TIA-Portal is an integrated development environment for configuration and programming of Simatic PLCs and Panels. TIA-Portal includes two former separate products, WinCC and STEP 7, and it integrates hardware configuration. STEP 7 is the part for programming controllers. WinCC is for programming panels. STEP 7 uses programming languages according to IEC 61131-3. The norm defines three graphical programming languages: ladder (LAD), function plan diagrams (FBD) and sequential function charts (GRAPH).

The user writes the program, stores it, compiles it and loads it into the PLC. The question is, how is the program stored? TIA-Portal has a long history. STEP 7 version 1 to 5 does not store the user input. It compiles the program in a way, which allows us to recreate a sort of user input. This compilation "destroys" the user input, but on the other hand, what you see is the semantic STEP 7 compiled. The biggest drawback of this strategy: if your input is not compile-clean, you cannot store.

TIA-Portal changed this strategy. Since version 10 STEP 7 stores the user program in a proprietary database. The user program is stored in many XML strings. These XML strings contain not only the user input, but they also contain data generated by syntax checking and early compilation steps. This extra information easily runs out of sync.

Customers want to preserve the user input – in case of a graphical programming language a drawing - not only in our data base. Modern software development also requires the usage of a version control system for the program code. Consequently, all user input must be in a textual format. Although textual representation of graphical programming languages is a contradiction, it is state of the art. There are two general ways to store a picture: bitmaps and vector graphics. For ladder, bitmaps or even ascii-art is possible, but would make the compiler a hard time. There are a bunch of file formats used for storing "pictures", just to mention two:

- DXF developed by Autodesk to interchange CAD data
- SVG is an XML based vector graphic format by the world wide web consortium

Graphical programming languages are not required to store a picture; they must store a graph. Like many electric design tools, for example

- LTSpice
- EAGLE
- KiCad

We need a format to store "compile-able drawings". These tools use textual representations of electrical schematics. This is a graph, and a graph has a set of nodes and a set of edges, which connect the nodes. Translated into their language: a node is a part; the set of nodes is the part list; a part has pins; the edges are wires; a wire connects pins of parts; the set of edges is the wiring list. This is the information these tools need for the electronic device described by the schematic and to simulate its behavior.

Node-RED is also a graphical programming language used for automation. <https://github.com/node-red/node-red/wiki/Flow-Format> describes the format for flows – the norm names them networks. And no wonder they talk about nodes and how they are connected by wires. Another graphical programming system is Labview by National Instruments, see https://en.wikipedia.org/wiki/LabVIEW#Graphical_programming. I quote the wiki "Nodes are connected to one another using wires".

Nodes are connected by edges. Parts are connected by wires. This is the basic information of all graphical programming languages; this is what the ladder and the function block diagram compiler need to translate the drawing into executable machine code.

1.1. First Example

To get the first expression let's start with an example. Don't ask if this makes sense, it's just an example.

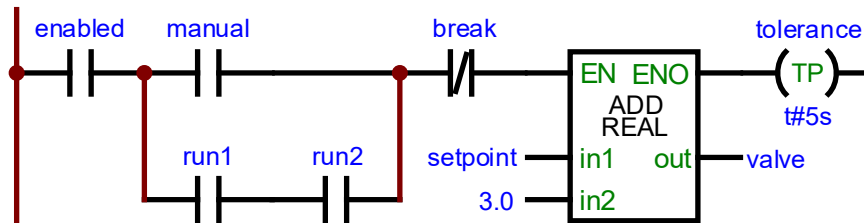


Figure 1 First Example

Figure 1 shows a small ladder network. Not in the colorless way TIA-Portal editor would display it. The colors are used for description in this document. The network has a Boolean expression, a mathematical expression and a timer coil. The semantic of all of this will be described in later chapters.

```
{ S7_Language = "LAD" }
NETWORK
  RUNG wire#powerrail
    Contact( enabled )
    wire#w1
    Contact( manual )
    wire#w2
    I_Contact( break )
    { S7_Tempate := "ValueType := Real" }
    Add(
      in1 := setpoint,
      in2 := 3.0,
      out => valve
    )
    tolerance.TP_Coil( t#5s )
  END_RUNG
  RUNG wire#w1
    Contact( run1 )
    Contact( run2 )
  END_RUNG wire#w2
END_NETWORK
```

Listing 1 First Example

Listing 1 shows the textual representation of Figure 1. I colored the syntax of Listing 1 according to Figure 1, this is not what a future syntax highlighting will do. There are four keywords. `NETWORK` and `END_NETWORK` are used to mark the beginning and end of a network. `RUNG` and `END_RUNG` are used to mark begin and end of a rung. For the moment, a rung is a horizontal line. A better definition will follow. This network has two horizontal lines in the graph – two rungs in its textual representation. The first rung starts with the left power rail. `wire#powerrail` is a predefined identifier. All well-formed ladder networks start at the left power rail.

Rungs are the core concept of ladder. All contacts, coils and boxes have one rung-in pin and one rung-out pin. The rung-in pin of the next element is connected to the rung-out pin of the previous element. The rung starts at a wire and in many applications, but not in TIA-Portal, it must end in a wire.

In the example this can be seen in the second rung. See Figure 1, the second rung begins with a wire `wire#w1` and ends in `wire#w2`. It has two contacts. The rung-in of the left contact is connected to `wire#w1`. The rung-out of the left contact is connected to the rung-in of the right contact. The rung-out of the right contact is connected to `wire#w2`.

This first example gives you an impression about Ladder textual format. Those of you familiar with ladder can understand the listing. Of course, there is syntax to learn, and programming languages have dozens of exceptions. As always there is a bunch of terminology to be understood.

1.2. IEC 61131-3

IEC61131-3: "The norm specifies the syntax and semantics of programming languages for programmable controllers. It specifies the syntax and semantics of a unified suite of programming languages for programmable logic controllers (PLCs). This suite consists of the textual language Structured Text (ST), and the graphical languages, Ladder Diagram (LD) and Function Block Diagram (FBD). An additional set of graphical and equivalent textual elements named Sequential Function Chart (SFC) is defined for structuring the internal organization of programs and function blocks. Also, configuration elements are defined which support the installation of programmable controller programs into programmable controller systems."

The current edition 4 has 245 pages. Chapter 7 describes in 8 pages structured text. Chapter 8 (12 pages) is about graphical languages and only chapter 8.2 with 5 pages is specific for ladder. The most pages (189) are used for the common elements which are used in all languages. The norm does not specify a textual representation; there are only a few examples in ASCII art.

Annex A (8 pages) gives a formal specification, a grammar, for the common elements, but only 6 lines mention ladder and function block diagrams:

```
Func_Body      : Ladder_Diagram | FB_Diagram | Stmt_List | Other_Languages;
FB_Body        : SFC | Ladder_Diagram | FB_Diagram | Stmt_List | Other_Languages;
Ladder_Diagram : LD_Rung *;
LD_Rung        : 'syntax for graphical languages not shown here';
FB_Diagram      : FBD_Network *;
FBD_Network     : 'syntax for graphical languages not shown here';
```

IEC chapter 8.2: "Ladder enables the programmable controller to test and modify data by means of standardized graphical symbols. These symbols are laid out in networks in a manner similar to a 'rung' of a relay ladder logic diagram. Networks are bounded on the left and optional right by power rails."

The norm talks about horizontal and vertical link elements. They represent what other named wires. The left power rail is always on – true. It talks about contacts, coils, functions and function blocks.

Contacts are shown as

```
***
--| |--
```

Coils are shown as

```
***
--( )--
```

Functions and function blocks are shown as boxes.

```

+-----+
X      | myFct |
-----| |-----| IN      |
+-----+
```

IEC: " networks shall be evaluated in top to bottom order as they appear in the ladder diagram "

Conclusion: the norm defines the semantic of ladder but specifies no textual representation.

1.3. STEP 7

STEP 7 is a part of TIA-Portal used for programming Simatic PLCs. STEP 7 has editors to create data types, data blocks, tag tables, and code blocks. These blocks are called program organization units by the norm.

STEP 7 and the norm distinguishes two types of code blocks: functions and function blocks. In a function blocks you can declare variables which keep their value to the next call. To call a function block you must define an instance. Functions cannot keep values to the next call; there is no instance.

The norm defines programs as function blocks. A configuration defines the instances for them and when the operating system will call them. There is no way to handover information from the operating system to the program. The norm defines only three event types. Simatic PLCs support more than 20 different event types. In Simatic PLCs the operating system is calling organization blocks. OBs are functions with an interface specified by the calling event. This way the operating system can hand over information to the application. But a function does not have an instance.

Code blocks have an interface, which can be described as specified in the norm. The body of these blocks can be written in many programming languages. STEP 7 allows to mix programming languages. Each network can be written in another language. STEP 7 allows to mix LAD, FBD, AWL, SCL inside one code block.

The library of STEP 7 has more instructions as proposed by the norm. The norm only defines a small core of instructions. There is a fistful of syntactical differences between norm instructions and instructions STEP 7 offers. But there is no functional limitation – no instruction is missing.

Currently this cannot be said about the data types, since Simatic does not support the data types `USTRING` and `UCHAR` which were introduced with edition 4.

Today STEP 7 does not support object-oriented programming extensions of the norm.

LAD is the STEP 7 version of the norms ladder. It has restrictions in comparison to the norm, e.g. STEP 7 does not support crossing wires or feedback by wiring. STEP 7 LAD is not a real power-flow compiler, see **Error! Reference source not found..** These restrictions are in respect to what can be expressed in one network. You can overcome these restrictions by splitting into several networks.

FBD is the STEP 7 version of the norms function block diagram. It has restrictions in comparison to the norm, e.g. STEP 7 does not support crossing wires, feedback by wiring, non-Boolean wiring. These restrictions are in respect to what can be expressed in one network. But you can overcome these restrictions by splitting into several networks.

SCL is the STEP 7 version of the norms structured text. ST code can be copied into the SCL editor and will be executable code if it does not contain the generally not supported features – UTF-8 encoded strings and chars, object-oriented programming.

1.4. Goals

The user wants to program a PLC with the help of LAD or FBD. This graph must be stored. The user wants to

- copy ladder and function block diagram
- undo changes
- get the difference to before the change
- collaborate -- exchange networks with another user
- use automated program generation

All this can be done easily with any textual representation, if the text can be put into a version control system. This will yield

- change detection / comparison
- merging
- collaboration

STEP 7 gets an improved

- Open-ness
- Library Handling
- Multiuser support

To achieve this this representation needs to be

- UTF-8 encoded
- line based
- foster short lines
- have a simple syntax
- small changes in the drawing shall lead to small changes in the text
- contain only information about the semantic of the network

The title mentioned "human readable". Why is "human readable" missed in the list before? Because the networks users created in the past are complex. Since a complex programming task results in a complex program. Graphical programming languages help to handle the complex Boolean logic. But complex Boolean logic is never human readable in textual representation. Complex Boolean logic is easier to comprehend in graphical representation.

The textual representation shall "contain only the information about the semantic of the network". This means no graphical positions, since STEP 7 has not a free editor like node-red does. This means no multilingual comments, since this would clutter the text. A program of an international working company using English, German, Mandarin, Afrikaans, French, Italian, Korean, ... would have dramatically more comments than compiler relevant lines – it would be unreadable.

Today's STEP 7 LAD editor has limitations. And there are good reasons for those limitations. But it is important that the syntax, the grammar are not restricted. It is not the goal to cement today's limitations. Everything that is covered by IEC 61131-3 must be expressed by the textual representation of LAD and FBD.

The textual representation shall enable

- crossing wires
- real power-flow networks
- object oriented features

2. General Concepts

LAD networks are parts of code blocks. IEC grammar is used to describe the interface of the block, data types, and literal constants.

TIA-Portal supports multilingual texts, a feature the norm does not specify. But the norm has a loophole: pragmas. Features not supported by the norm are mapped to pragmas.

Our documents are stored in UTF-8 beginning with the byte order mark sequence `ï»¿` (`0xEF 0xBB 0xBF`). With this byte order mark, many text editors like notepad++, WordPad, Code, ..., know that the file is encoded as UTF-8. With that restriction we don't have to deal with local Windows settings and different codepages.

But STEP 7 does also accept text files without the byte order mark sequence, but it will be treated with the current operating system code page. If this text is changed and written by STEP 7, the encoding is UTF-8, and the byte order mark sequence is re-created.

.Net `System.String` uses UFT-16. But most of the fonts shipped for Windows do not contain surrogate pairs. This means the chance that a document contains a surrogate pair is almost 0. Hence, we decided to forbid the surrogates, the private planes, and the code points above `16#E000`. Thus, all character from `16#D800` .. `16#FFFF` are excluded. We can decide to allow some of the codepoints `16#F900` .. `16#FFFD`, but that depends on the requirements and the compatibility with older TIA versions.

Non-printable characters `16#0000` .. `16#001F` and `16#007F` .. `16#00A0` are excluded, while of course `CR` (`u000D`), `LF` (`u000A`), `FF` (`u000C`) and `TAB` (`u000B`) are allowed.

Because of historical reasons STEP 7 has two different mnemonics that play a role in the representation of absolute addresses and STL operations.

Ladder textual format will always use international (English) mnemonic.

The detections of keywords like elementary data types, array types, structure types and block and section delimiters are case insensitive. Also, the mapping between usage of variables in the block body and their declarations is case insensitive. The formats for types in the lexical rules are case insensitive but we will show that explicitly.

STEP 7 is case insensitive but case preserving. This also applies to defined pragmas.

2.1. Blocks

STEP 7 blocks are entities of a user program delimited according to their function, their structure, or their intended use. STEP7 allows you to program and use the following types of blocks:

- OB
- FC / SFC
- FB / SFB
- DB / SDB
- UDT / SDT
- FBT

All blocks consist of the following components with defined order:

- Start/end of block header (keyword corresponding to block type)
- Block attributes (optional)
- Block header
- Block body (start values in the case of data blocks)

When a block is exported STEP 7 generates a declaration file with the ending `s7dc1`.

2.2. Constants

For literal constants the syntax defined by the norm is used.

2.3. Interfaces

For the interface of a block the syntax defined by the norm is used.

2.4. Networks

A code block must have at least one network. The network is introduced by the keyword `NETWORK` and terminated with `END_NETWORK`. Each network can be written in a different programming language. The block does not have a programming language, but the editors have a default for new networks, the so-called preferred language.

The mandatory pragma `{ S7_Language := "programming_language" }` specifies the programming language for the network. This pragma must be used for networks. It can be used only at network level. A network without any defined programming language is invalid.

Currently the following programming languages are supported.

- STL
- LAD
- FBD
- SCL

```
FUNCTION_BLOCK Motor
  VAR_INPUT
    Input1 : INT
  END_VAR
  ...
  { S7_Language := "SCL" }
  NETWORK
    #Value1 := #Input1 + 10;
  END_NETWORK
  { S7_Language := "LAD" }
  NETWORK
    RUNG wire#powerrail
      Move(
        in := #Input1,
        out => #Value1
      )
    END_RUNG
  END_NETWORK
END_FUNCTION_BLOCK
```

Listing 2: A function block with different network languages

Listing 2 shows a function block with two networks. The first is written in SCL, the second is written in LAD.

3. Ladder Concepts

Ladder is a graphical programming language. A graph is a set of nodes and a set of edges. The nodes represent functions and operations. The nodes are the instructions of the programming language. Every programming language needs instructions. There are inherent instructions – the core, the stuff that just exists, falls from the sky. And there are libraries where someone else provided, with the help of the inherent stuff, something for you that you can use to write your application comfortably. These libraries save you a lot of time.

The edges between the nodes specify the data flow. Variables and constants can be connected to inputs of nodes. And the outputs of nodes can be connected to variables or inputs of other nodes. As in all programming languages, the data type of the source must fit to the sink. You cannot store a string in a variable of type Bool.

The data flow in LAD is from left to right. The norm does specify this for nodes, but not for wires.

The following chapters define the base to understand the textual representation of ladder. What is possible, available in the graphical representation and how this is mapped to the textual representation. We need to talk about the library, the inherent elements to understand this mapping. Although the user cannot define these elements.

3.1. Logic

Ladder is intended for programming Boolean expressions. It offers all kinds of instructions, mathematical, string functions and program control like all other programming languages. But it is best for Boolean. Interestingly it does not offer logic boxes. Instead, the wiring of contacts is used to express the Boolean functions AND and OR.

Contacts represent switches of relays. The operand on top of the contact determines if the switch is on or off. If it is on it can conduct electricity. Two contacts in a row represent a logical AND. Two contacts in parallel represent a logical OR.

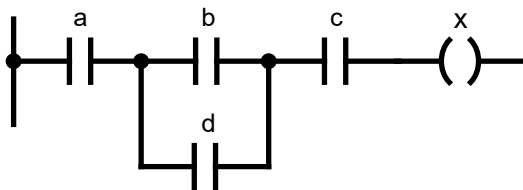


Figure 2 Boolean Logic in LAD

Figure 2 represents a Boolean OR with two inputs and a Boolean AND with three inputs. It represents the Boolean expression $x := a \text{ AND } (b \text{ OR } d) \text{ AND } c$. The two contacts for b and d are connected in parallel. It does not matter which is on to conduct. The vertical line on the left side is considered to provide electric power. To energize coil x both contacts a and c and at least one of b or d must be on.

Two contacts in parallel act as Boolean OR. Two contacts in series act as Boolean AND.

The gentle read may notice that $a \text{ AND } (b \text{ OR } d) \text{ AND } c$ needs a pair of brackets to achieve the semantic of Figure 2. AND has precedence over OR. Normal sized networks with 30 contacts are easy to handle, especially in debugging sessions. Boolean expressions in SCL with 15 pairs of brackets are not. This advantage, no need for brackets, is the unbeatable reason to use graphical programming languages for Boolean expressions.

3.2. Gates

Figure 1 shows four contacts, a box and a coil. They all represent functions, they are instructions from the inherent system library, they are gates.

A gate is a library element, representing a function, representing an instruction, having a name and a graphical representation. In the context of LAD, instructions and gates are synonyms.

Ladder has only a small set of graphical representations of gates:

- Contacts
 - Small contacts
 - Big contacts
- Coils
 - Small coils
 - Big coils
- Boxes
 - ENO boxes
 - Q boxes
- Labels

But the inherent library has about 230 instructions. Details can be found in ####. Every gate has a different function, represents a different instruction. Inside the graphical representation the gate shows its name. The name specifies its function, its instruction. The box shows if it is a Add, Mul, Div, ... and so on.

The gate name is use in the textual representation of LAD to identify the instruction. The textual representation reuses the syntax of a function call. Every instruction gets its own line in the text, to support the version controls system's change detection.

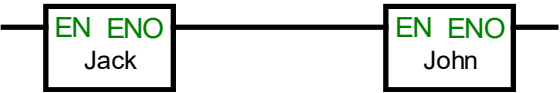


Figure 3 Two Instructions

```
Jack ()
John ()
```

Listing 3 Two Instructons

Figure 3 and Listing 3 show a very simple network fragment with two instructions named Jack and John.

The gate name is used to identify the gate. The importer will see Jack followed by (. So, Jack should be the name of an inherent instruction or a code block of the system library or a code block of user program and the editor will show a box for Jack.

First it searches for Jack in the list of keywords. Then it searches for Jack in the local variables, then in the inherent library, then in all opened system and user libraries. And finally, it searches in the project to find a user block of this name. If it does not find Jack, or Jack is not a user function or instruction, Jack is shown red. But due to the following (, Jack is always identified as instruction.

3.3. Pins

An instruction, function, gate has an interface. A pin is the graphical representation of a parameter. A pin is a parameter it has a name and a data type. Most times the parameter names are shown inside the box.

Most times (see 3.3.1) input-pins are shown to the left of the box; output-pins are shown to the right of the box. Most times (see 3.3.3) in-out-pins are shown to the left of the box, but the parameter name is preceded with $\rightleftharpoons$ to indicate that data flows in both directions.

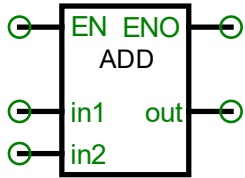


Figure 4 Add Box

Figure 4 shows an Add-box with the pins of this box. The green circles indicate positions where things can be connected to. The editor will not show but obey them. Depending on the parameters data type variables, constants or wires can be connected to the pins.

```
Add (
    in1 := ,
    in2 := ,
    out =>
)
```

Listing 4 Add Box

Listing 4 shows a textual representation of the Add-box of Figure 4.

This is the same syntax as used for structured text. Input- and in-out-parameter are according to the norm assigned with `:=`, outputs are assigned with `=>`. All parameters are and must be listed. This avoids additional pragmas in the case of boxes with a variable number of parameters. An Add-box with four inputs is represented as

```
Add (
    in1 := a,
    in2 := 4,
    in3 := ,
    in4 := d,
    out => sum
)
```

Listing 5 Add Box With Four Inputs

In Listing 5 the parameter `in1` is connected to something named `a`, `in2` is connected to literal `4`, `in3` is not connected, `in4` is connected to something named `d` and `out` is connected to something named `sum`.

If a parameter is missing, e.g. `in2`, the import fails. If `a`, `d` and `sum` exist do not play a role for the import.

Most of the boxes start numbering with 1, but there are exceptions like the select box starting with 0.

Every parameter gets its own line in the textual representation, to foster change detection in a version control system.

When the parameter is connected to a variable or constant, the name of the connected object is found in the line of the parameter.

In the case of contacts and coils all parameters play specific roles. The pin names are not shown, and the connected constants or variables can be shown above or below the contact, coil, or box, see below.

3.3.1. Rung-In, Rung-Out

Every ladder instruction, every gate has a rung-in pin. Most ladder instructions have a rung-out pin. Jumps do not have a rung-out. And the label is the exception for having no rung-in, see 4.6.

Only one pin can be marked as the rung-in. Only one pin can be marked as the rung-out. Rung-in and rung-out must be different pins of data type Bool. These pins are used to build rungs, see 3.7.

Rung-in and rung-out parameters are not listed in the textual representation. This reduces the number of parameters in the textual representation. In the case of normally open contact, only one parameter remains. If only one parameter must be listed, we will use the same line for the parameter as the instruction.

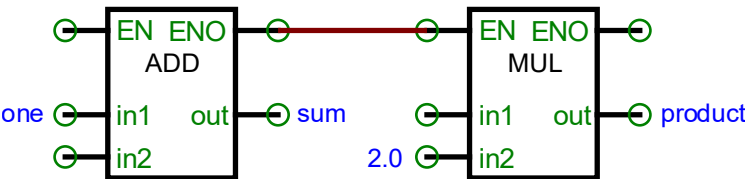


Figure 5 Rung-Out to Rung-In

```
Add (
  in1 := one,
  in2 := ,
  out => sum
)
Mul (
  in1 := ,
  in2 := 2.0,
  out => product
)
```

Listing 6 Rung-Out to Rung-In

Listing 6 is the textual representation of Figure 5. It shows no `en` and `eno` for the math boxes. Although these pins are parameters of the instructions.

3.3.2. Operand

One parameter of an instruction can be marked with text specific role `operand`. In this case neither the parameter name nor the assignment is exported.

The instructions for contact and coils have three parameters:

Parameter Name	in	operand	out
Data Type	Bool	Bool	Bool
Lad Role	rung-in	top	rung-out
Text Role		operand	

Table 1 Contact Parameters

`in` is the rung-in and `out` is the rung-out of a contact. There is just one parameter left, so we will write it into the same line as the instruction. Parameter `operand` is marked as `operand` hence we will neither write the parameter name `operand` nor `:=`.

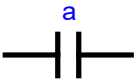


Figure 6 Contact

456 Instead of

```
457     Contact (
458         operand := a
459     )
```

460 We will export

```
461     Contact ( x )
```

462
463 If the contact is not connected to an operand, then we will just see

```
464     Contact ( )
```

465

466 A simple graphical element has a simple textual representation. This helps to avoid text where no ambiguities are
467 expected. This helps to achieve the goals, see 1.4.

468

469 **3.3.3. Top, Bottom**

470 The operand of contacts and coils has the ladder specific role `top`. The has two effects. The editor will neither show a pin
471 for the parameter nor the name of the parameter – there is no space for it in the graphical representation. But the editor
472 will show an input-field on top of the graphical representation and allow to write something into it. This will be connected
473 to the parameter.

474 This feature is not restricted to contacts and coils it can also be used with boxes. For example, the flipflops (see 4.5.1)
475 show their storing element above the box.

476

477 Some instructions have one parameter with the ladder specific role `bottom`. The has two effects. The editor will neither
478 show a pin for the parameter nor the name of the parameter – there is no space for it in the graphical representation. But
479 the editor will show an input-field below of the graphical representation and allow to write something into it. This will be
480 connected to the parameter.

481

482 Only one parameter can be marked as `top` parameter, only one parameter can be marked as `bottom` parameter. This
483 must be different parameters. Being a `top` or `bottom` parameter has no influence on textual representation.

484

485 **3.3.4. Instance**

486 An instruction parameter can be marked with the text specific role `instance`. This has no effect on the editor, on the
487 graphical representation, but on the textual representation. If a parameter is marked as `instance`, the textual
488 representation looks like

```
489     Instance.Instruction ( )
```

490

491 Timer (see 4.5.2) and Counter (see 4.5.3) use this feature, and calls to function blocks are represented this way (see
492 4.4.3).

493

3.4. Templates

Templates are used to specialize instructions.

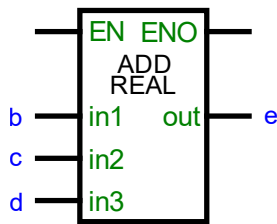


Figure 7 Add Real

Figure 7 shows an `ADD_REAL`. This means an `ADD`-box where the programmer has selected data type `Real`. The data type defines how the math box calculates, which defines how the overflows and the `eno` of the box is calculated. The given data type specializes the generic `ADD`. If a customer drags a math box from the instruction browser into a network, the editor will not select a data type. The box will show `Auto` as data type. When a variable is supplied to one of the inputs, the editor will show the type of the variable plus `Auto`. This means that the box currently uses the shown type, but the type can change when the type of variables connected to the inputs are changed. If the customer selects a data type in the drop down of the editor, the `Auto` will disappear. The variables lose their influence.

To specialize instructions, the library uses so-called templates.

Templates have a name and a value. If the value is different to the template names default, the textual representation must note the template. Templates are shown as pragmas according to the norm – this means inside curly brackets.

```
{ S7_Templates := "valueType := Real" }
Add(
    in1 := b,
    in2 := c,
    in2 := d,
    out => e
)
```

Listing 7 Add Real

Listing 7 shows the textual representation of Figure 7. To specify that this addition must use the data type `Real`, a pragma with name `S7_Templates` is exported before the instruction, since pragmas influence the following object – the `Add`.

The pragma name is followed by one or more key value pairs. In general, the pragma for a list of templates looks like

```
{ S7_Templates := "[ TemplateName := TemplateValue, ... ]" }
```

In case there is only one template, the square brackets can be omitted

```
{ S7_Templates := "TemplateName := TemplateValue" }
```

If the customer has selected a data type, then we will export a pragma, but we do not export "auto" in case of automatic data type selection.

There are instructions with several templates

```
{ S7_Templates := "[ inType := Int, outType := Real ]" }
Convert(
    in := x,
    out => y
)
```

3.5. Parts

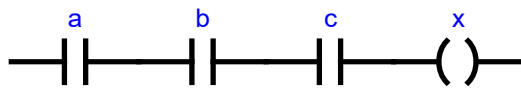


Figure 8 Parts

Figure 8 shows three contacts and a coil. It shows four parts. A part is a gate used in a network. A gate can be used many times in a network.

```

Contact( a )
Contact( b )
Contact( c )
Coil( x )

```

Listing 8 Parts

Listing 8 shows the textual representation for Figure 8 is as simple as possible. It shows the four instructions with their parameters. There are no parameter names, no syntactical sugar, no UIDs, no graphical positions.

Although this information exists all inside the editor and the cross-reference database, it is not relevant to express the semantic of the network fragment and hence it is not relevant for the compiler. Therefore, it is not in the textual representation. See the goals in 1.4.

The import considers each identifier followed by a (to be a part. If the identifier is found in the set of keywords, local variables, or global variables it's not usable as instruction. The editor will show a red box, with a red box name. If the identifier is an inherent library element the editor will show its graphical representation, a contact, coil or box. In all other cases, the editor will show a box, if library block or user block is identified the box is shown black, otherwise red.

Between the round brackets there should be the instruction parameters. Import will accept three situations

```

Instruction()                for instructions without parameters
Instruction( parametervalue ) for instruction with one parameter, remaining according to ####.
Instruction(                  for instructions with more than one parameter
After the last case import will accept another set of three situations
parametername := parametervalue, for in- and in-out parameter
parametername => parametervalue, for out parameter
)                                for the end of the parameter list

```

This allows the import to show an adequate box for the part with pins on left or right side even when the instruction is not identified. Inputs can be connected to variables or constants, outputs and in-outs can be connected to variables.

A successful import generates a mapping between the internal representation and the imported text. The internal representation has UIDs for every part and wire. The graphical representation adds in a first step additional device independent, grid-based x-y-positions to the parts. In a second step the editor adds device and zoom dependent positions to the parts.

3.6. Wires

Semantically, a wire is a connection between a set of pins. For the data model, a wire is a set of pins. A wire is the graphical representation of a data flow. STEP 7 allows only Boolean wires, but the textual representation does not care too much about that. All connected input-pins read the same value. All connected output-pins build a logically OR. This means a wire fulfills two tasks, it is a logical OR for its information sources, that transports its result to many information sinks.

Graphically, the wire is a set of horizontal and vertical lines. The norm allows crossing unconnected lines, so does Ladder textual format; STEP 7 does not. If a horizontal and a vertical line touch each other, a filled circle indicates that they are connected, they build one wire. If the filled circle is missing, the two line are not connected and represent two independent wires.

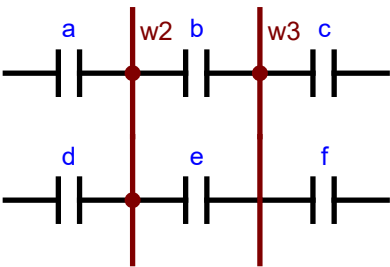


Figure 9 Connected and Unconnected Wires

Figure 9 show a situation according to the norm, which is not supported by STEP 7. The textual representation must be able to handle this. There are two wires. The open ends of the wire indicate that this is only a detail of a greater network. Wire `w2` connects at least the output pins of contacts `a` and `d`. Wire `w2` connects at least the input pins of contacts `b` and `e`. This is supported by STEP 7. Wire `w3` connects the output of contact `b` and the input of contact `c`, but not the output of contact `e` and the input of contact `f`. According to the norm the output of contact `e` is connected to the input of contact `f`.

```

Contact ( a )
wire#w2
Contact ( b )
wire#w3
Contact ( c )
```

Listing 9 Connected Wires

Listing 9 shows the textual representation of the upper part of Figure 9.

The pins marked as rung-in or rung-out are not listed by name, but their instructions are. The connection to wire `w2` between the contact with operand `a` and the contact with operand `b` in the textual representation is simply the name of the wire between the contact with operand `a` and the contact with operand `b`. The connection to wire `w3` between the contact with operand `b` and the contact with operand `c` in the textual representation is simply the name of the wire between the contact with operand `b` and the contact with operand `c`.

```

Contact ( d )
wire#w2
Contact ( e )
Contact ( f )
```

Listing 10 Unconnected Wires

Listing 10 shows the textual representation of the lower part of Figure 9. No surprise wire `w2` is listed between the contact with operand `d` and the contact with operand `e`. There is no wire `w3` listed between the contact with operand `e` and the contact with operand `f` – there is no connecting circle in the graphical representation.

To describe the situation, I used names for the wires. I could have used ids, that wouldn't make any difference. But, is this necessary? Wasn't the goal to avoid any ids? Short answer, it's not possible to avoid all ids. A graph is a set of nodes and a set of edges. The edges connecting the nodes. For ladder the nodes are the parts, and the edges connect pins of parts. A textual representation of a graph can solve the task in two ways. Either it names the parts, so the wire is a list of part-pin names, or it names the wires, so the pins store the wire they are connected to. Thanks to rungs, see below, the number of wire-ids is much smaller than the otherwise necessary number of part-ids.

It is possible to have another identifier, for example a local variable `w1` which is not a wire. How can wires be distinguished from any other named object, for example a local variable. Remember, it is possible to connect a wire to a parameter of an instruction.

```
Instruction(
    in1 := w1 // wire or variable ??
)
```

There are two ways to handle this. We could introduce a keyword or a scope. In LAD both solutions are equal. In FBD the scope has more benefits, see the discussion in ####.

A wire must be prefixed with `wire#`. The prefix avoids confusion with other local variables. The identifiers of wires are identifiers as any other identifiers. They must start with a Unicode character of category letter, followed by Unicode characters of category letter or digits, see **Error! Reference source not found.** If the user connects pins in a way a wire is created, the STEP 7 editor will name it `wire#w` followed by an integer number. If the user imports a network, the editor will keep the name of the wire until editing deletes the wire. The user is free to give the wires any name, it's just a string, following the restrictions of identifiers. STEP 7 will keep the user chosen names. This keeps the number of differences in the version control system low. Only real editing actions will create changes in the textual representation.

Names for wires have a network local scope. The identifier is valid only in the network where the wire is defined. The other way round, every network can have a `w1`, these are different wires.

According to the norm every network has a left power rail. The left power rail has the value true. The right power rail is optional. The left side power rail identifier is `wire#powerrail`.

According to the norm and many competitor implementations you can have a `wire#leftrail` and a `wire#rightrail`. These are accepted but not used by STEP 7.

3.7. Rungs

First but incomplete definition: A rung is a list, an ordered set, of instructions.

Chapter 3.3.1 introduced rung-in and rung-out pins. Most gates and so their parts have them. They are used to build rungs. But what does "building a rung" mean.

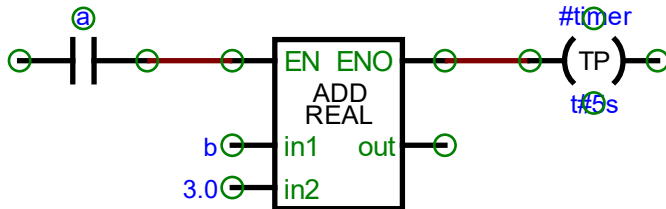


Figure 10 A Rung

Figure 10 shows three parts with their pins – the STEP 7 editor wouldn't do that. To emphasize the fact that the rung-out pin of the contact with operand a is connected to the EN parameter, of the Add-box Figure 10 shows a red line. It also shows a red line between the ENO of the Add-box and the TP-coil. The STEP 7 editor does not emphasize those lines.

```

RUNG
  Contact( a )
  Add(
    in1 := b,
    in2 := 3.0,
    out :=
  )
  TP_Coil(
    timer := #timer,
    pt := t#5s
  )
END_RUNG

```

Listing 11 A Rung

Listing 11 shows the textual representation of Figure 10. You see two keywords: RUNG and END_RUNG. They define a list – the order of the elements in this list is important. The order in the list represents the data flow in the graph.

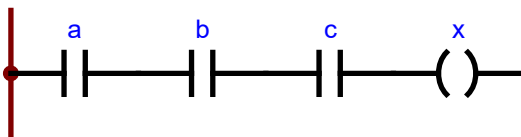


Figure 11 A Semantical Correct Network

Figure 11 shows the first semantically correct network in this chapter. The vertical line at the left side is the power rail. This is the ladder equivalent to following SCL statement: `x := a and b and c;`

```

NETWORK
RUNG wire#powerrail
  Contact( a )
  Contact( b )
  Contact( c )
  Coil( x )
END_RUNG

```

```
END_NETWORK
```

Listing 12 A Semantical correct Network

Listing 12 shows the textual representation of Figure 10. The gentle reader may notice, the rung has got an operand, the power rail wire.

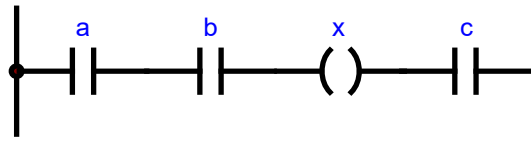


Figure 12 The Order Of Instructions

```
NETWORK
```

```
  RUNG wire#powerrail
```

```
    Contact( a )
```

```
    Contact( b )
```

```
    Coil( x )
```

```
    Contact( c )
```

```
  END_RUNG
```

```
END_NETWORK
```

Listing 13 The Order Of Instructions

Figure 12 and Listing 13 demonstrate the order matters. Graphically and syntactically the rung is ok, but semantically contact with operand c is useless. The rung represents in SCL just `x := a and b`. The last contact would be ignored. The STEP 7 compiler will reject this network with an error message that the rung should end with a coil or box.

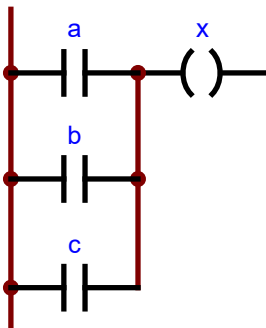


Figure 13 Vertical Wires

Figure 13 shows three contacts connected in parallel. There are two vertical lines – two wires.

```
NETWORK
```

```
  RUNG wire#powerrail
```

```
    Contact( a )
```

```
    wire#w1
```

```
    Coil( x )
```

```
  END_RUNG
```

```
  RUNG wire#powerrail
```

```
    Contact( b )
```

```
  END_RUNG wire#w1
```

```
  RUNG wire#powerrail
```

```
    Contact( c )
```

```
  END_RUNG wire#w1
```

```
END_NETWORK
```

Listing 14 Vertical Wires Require Several Rungs

743 Listing 14 shows the textual representation of Figure 13. All rungs start with `wire#powerrail`. Graphically, the first
744 rung ends with a coil. In the text, the coil is the last instruction, the first rung ends without a wire. The other rungs end
745 with `wire#w1` as they do in the graph.
746

747 The left most wire in ladder is always the power rail. Logically the power rail is always true. The input pins of three
748 contacts are connected to the power rail. The three outputs are connected to `wire#w1`. They build a logical OR. This result
749 is connected to the input of the coil.
750 Staying with the historical picture, if one of the contacts is closed, the second wire gets true and the coil is energized.
751 Only if all three contacts stay open, the second wire is false and the coil is not energized.
752

753 In Ladder rungs can start and end with wires. In compile-able networks all rungs start with wires.
754

755 Rungs represent horizontal lines in the graph.
756 Vertical wires connect rungs. In most scenarios wires represent vertical lines in the graph.
757 But when connected to remaining Boolean parameters of instructions, then the wire could look like a horizontal line, see
758 an example in 4.5.1. But they are still connecting different rungs.
759

760 Second definition: In LAD a rung is a list, an ordered set, of instructions or wires.
761

3.8. Summary

Let's repeat the important definitions of this chapter:

- Instructions and gates are synonyms. A gate, an instruction has a name.
- A gate is a library element, representing a function, an instruction.
- A pin is the graphical representation of an instruction parameter.
- Pins can have ladder specific and text specific roles. Roles define how pins can be connected. Ladder specific roles define how they are represented in the graph. Text specific roles define how they are represented in the text.
- A part is a gate used in a LAD network.
- A wire is a set of pins. All connected output-pins build a logical OR. All connected input-pins read the same value.
- A wire has a name (identifier) which is valid only in the network it is listed. It is scoped, prefixed with `wire#`. `wire#powerrail` is a predefined name.
- A rung is a list of instructions or wires. It can start and end with a wire.
- A LAD network is a list of rungs.

The lines of ladder networks will all look like (the **mandatory** parts in bold, the *optional* in italics):

NETWORK	begin of a network
END_NETWORK	end of a network
RUNG <i>wire</i>	begin of a rung
END_RUNG <i>wire</i>	end of a rung
<code>Instruction()</code>	instruction without remaining parameter
<code>Instruction(symbol)</code>	instruction with one remaining parameter
<code>Instruction(</code>	instruction with more than one remaining parameter
<code>Parametername := <i>parametervalue</i>,</code>	in- or in-out parameter
<code>Parametername => <i>parametervalue</i>,</code>	out parameter
<code>)</code>	instruction end
<code>{ Pragmaname }</code>	single pragma without value
<code>{ Pragmaname := pragmavalue }</code>	single pragma with value
<code>{</code>	begin of a list of pragmas
<code>Pragmaname;</code>	pragma without value in a list
<code>Pragmaname := pragmavalue;</code>	pragma without value in a list
<code>}</code>	end of pragma list

Most of the grammar is the grammar of IEC 61131-3.

Pragma values can be

<code>true</code>	Boolean value
<code>false</code>	Boolean value
<code>integer value</code>	Integer Values
<code>"symbols"</code>	allowed single value for pragma name
<code>"[symbols, ...]"</code>	list of allowed values for pragma name
<code>"templatename := templatevalue"</code>	allowed key-value pair for pragma name
<code>"[templatename := templatevalue, ...]"</code>	list of allowed key-value pairs for pragma name
<code>"templatename := [templatevalue, ...]"</code>	list of allowed values for key of pragma name

4. Instructions

In 3.1 was mentioned that LAD has just a fistful of graphical representation for all the instruction it supports. These representations kind of classify the more than 200 inherent instructions since they are handled very similarly. Hence it is not necessary to describe them all here.

There are contacts, coils, ENO-boxes, Q-boxes, and oddballs like the NOT and the label.

4.1. Contacts

Ladder uses contacts to read Boolean variables. There are no other Boolean instructions like AND, OR, or XOR. A contact is a Boolean AND with two inputs, the first is named `in`, the second `operand`.

The normal `Contact` and the inverting `I_Contact` have one operand. But LAD has contacts with implicit operands, and with two operands.

According to the norm there are

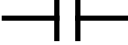
Representation	Instruction		Semantic
	<code>Contact(...)</code>	normally open	<code>out := in AND operand</code>
	<code>I_Contact(...)</code>	normally closed	<code>out := in AND NOT operand</code>
	<code>P_Contact(...)</code>	positive edge	<code>out := in and not(bit); bit := in;</code>
	<code>N_Contact(...)</code>	negative edge	<code>out := not(in) and bit; bit := in;</code>

Table 2 Inherent Contacts

4.1.1. Contacts With One Operand

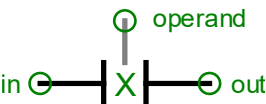


Figure 14 Contact with One Operand

Figure 14 shows an X for the instruction implemented by the contact. X is a space for normally open contacts and a slash for the normally closed, inverted contacts. `in` is the rung-in pin and `out` is the rung-out pin. The `operand` input has the text specific role `operand`, see 3.3.2, and the ladder specific role `top`, see 3.3.3.

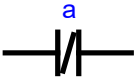


Figure 15 Nomally Closed Contact

```
I_Contact( a )
```

Listing 15 Nomally Closed Contact

Listing 15 is the textual representation of Figure 15. Since there is only one remaining parameter it's written at the same line as the instruction and since the parameter has the text specific role `operand` the parameter name is not exported (see 3.3.2).

STEP 7 supports a bunch of additional contacts with one operand.

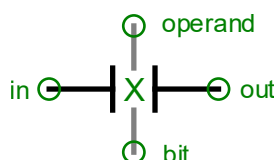
`IsValidContact`, `IsValidContact` has only one parameter and is used to test if a variable of a floating-point data type does not or does have the values `+NaN`, `-NaN`, `+Inf`, or `-Inf`. `isArrayContact`, `isArrayContact` has only one parameter and is used to test if a variable of data type `Variant` does or does not have address and array.

`IsNullContact`, `IsNullContact` has only one parameter and is used to test if a variable of reference data type does or does not have the value `Null`.

847

848 4.1.2. Contacts With Two Operands

849



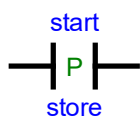
850

851 Figure 16 Contact with Two Operands

852

Figure 16 shows an X for the instruction implemented by the contact. X is a P for positive, raising edge detection contact and a N for negative, falling edge detection contact. `in` is the rung-in pin and `out` is the rung-out pin. The `operand` input has the text specific role `operand`, see 3.3.2, and the ladder specific role `top`, see 3.3.3. Since edge detection needs a storing element, these contacts have an addition in-out named `bit`, with the ladder specific role `bottom`.

857



858

859 Figure 17 Edge Detection Contact

860

```

861 P_Contact(
862     operand := start,
863     bit := store
864 )

```

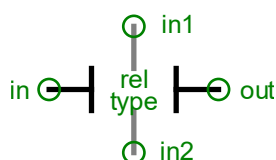
865 Listing 16 Edge Detection Contact

866

Listing 16 is the textual representation of Figure 17.

868

869



870

871 Figure 18 Compare Contact

872

Figure 18 shows the graphical representation of comparisons for the 6 relations: `EQ`, `NE`, `LT`, `GT`, `LE`, and `GE`.

Inside the contact you see the relation of the comparison and the data type used for the comparison. The user can choose or change the relation and the data type.

```

876 { S7_Templates := "[ SrcType := DInt ]" }
877 LE_Contact(
878     in1 := ,
879     in2 :=
880 )

```

881 Listing 17 Compare Contact

882

883 Listing 17 is the textual representation of Figure 18. The relation is reflected in the instruction. The data type is noted as

884 template pragma.

885

886 STEP 7 supports a bunch of additional contacts with two operands but without a data type template.

887 EQ_TypeContact and NE_TypeContact can be used to compare data types, to check for the data type of a variable of

888 data type Variant. One input should be connected to a variable of data type Variant; the other input should be a data

889 type literal (for example Real).

890 EQ_ElemTypeContact and NE_ElemTypeContact can be used to check array elements data type, in cases where the

891 variable of data type Variant is an array.

892 EQ_TypeOfDBContact and NE_TypeOfDBContact can be used to check data types of data blocks, in cases where the

893 variable is of data type DB_ANY.

894

895 **4.1.3. Contacts Without Operands**

896 Does a contact without an operand make sense? What is the partner for the logical AND? The instruction is giving the

897 operand implicitly.

898

899 For the support of AS300 and AS400 we add contacts for accessing the status word flags. They look like one operand

900 contacts; the flags expression is shown at top position. But these contacts are instructions; you must change the

901 instruction, not the operand. Table 3 shows the legacy contacts, which are only available if the network is intended to be

902 executed on AS300 or AS400.

903

	flags expression
BR_FlagContact()	BR
OS_FlagContact()	OS
OV_FlagContact()	OV
EQ_FlagsContact()	not A0 and not A1
NE_FlagsContact()	A0 <> A1
GE_FlagsContact()	not A0
LE_FlagsContact()	not A1
GT_FlagsContact()	not A0 and A1
LT_FlagsContact()	A0 and not A1
UO_FlagContact()	A0 and A1

	flags expression
BR_I_Flag_Contract()	not BR
OS_I_Flag_Contract()	not OS
OV_I_Flag_Contract()	not OV
EQ_I_Flags_Contract()	A0 or A1
NE_I_Flags_Contract()	A0 = A1
GE_I_Flags_Contract()	A0
LE_I_Flags_Contract()	A1
GT_I_Flags_Contract()	A0 or not A1
LT_I_Flags_Contract()	not A0 or A1
UO_I_Flag_Contract()	not A0 or not A1

904 Table 3 Legacy Contacts

905

4.2. Coils

Ladder uses coils to write Boolean variables. There are 33 coils in the inherent library. Coils with no, one, and two parameters. For all coils, the line-in value from the left is copied to the line-out on the right.

According to the norm there are

Representation	Instruction		Semantic
	Coil(...)	coil non-inverting	out := in; operand := in;
	I_Coil(...)	negated coil inverting coil	out := in; operand := NOT in;
	S_Coil(...)	set coil	out := in; IF in THEN operand := TRUE; END_IF;
	R_Coil(...)	reset coil	out := in; IF in THEN operand := FALSE; END_IF;
	P_Coil(...)	positive edge coil	out := in; operand := in and not(bit); bit := in;
	N_Coil(...)	negative edge coil	out := in; operand := not(in) and bit; bit := in;

Table 4 Inherent Coils

4.2.1. Coils With One Operand

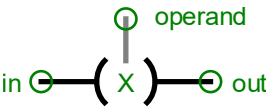


Figure 19 Coils with One Operand

Figure 19 show an X for the instruction implemented by the coil. X is a space for non-inverting coils and a slash for the inverting coils. X is an S for the coil that can only set the operand and an R for the coil that can only reset the operand. in is the rung-in pin and out is the rung-out pin. The operand input has the text specific role operand, see 3.3.2, and the ladder specific role top, see 3.3.3.

STEP 7 supports a bunch of additional coils with one operand. The parameter name is not always operand to emphasize the relation to counter and timer boxes.

Instruction	Parameter Name	
CU_Coil(...)	counter	count up
CD_Coil(...)	counter	count down
RT_Coil(...)	timer	reset timer
JumpCoil(...)	label	see 4.6.4
I_JumpCoil(...)	label	see 4.6.4
ReturnCoil(...)	operand	see 4.6.2
CallCoil(...)	operand	see 4.6.1
OpenDBCoil(...)	operand	open data block; only AS300, AS400
OpenDICOil(...)	operand	open data block; only AS300, AS400

Table 5 Additional One Operand Coils

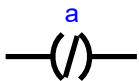


Figure 20 Inverted Coil

```
I_Coil ( a )
```

Listing 18 Inverted Coil

Listing 18 is the textual representation of Figure 20. Since there is only one remaining parameter it's written at the same line as the instruction and since the parameter has the text specific role `operand` the parameter name is not exported (see 3.3.2). So, it doesn't matter that the parameter name is not always `operand`.

4.2.2. Coils With Two Operands

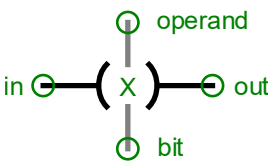


Figure 21 Coil with Two Operands

Figure 21 shows an X for the instruction the coil implements. X is a P for positive, raising edge detection coil and a N for negative, falling edge detection coil. The edge detection coils set the operand true for only one execution – when the edge at the input `in` is detected. `in` is the rung-in pin and `out` is the rung-out pin. The `operand` input has the text specific role `operand`, see 3.3.2, and the ladder specific role `top`, see 3.3.3. Since edge detection needs a storing element, these contacts have an addition in-out named `bit`, with the ladder specific role `bottom`.

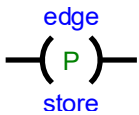


Figure 22 Positive Edge Coil

```
P_Coil (
  operand := edge,
  bit := store
)
```

Listing 19 Positive Edge Coil

Listing 19 is the textual representation of Figure 22.

959 Timer and counter can be controlled by two operand coils. AS1500 supports timers and counters according to the norm
 960 and for compatibility reasons the timers and counter of the AS400. In respect to the textual representation this does not
 961 make a difference. There is a parameter named `timer` and a parameter for the preset time named `pt`.
 962

Instruction	First Parameter	Second Parameter	
TP_Coil	timer	pt	start pulse
TON_Coil	timer	pt	start on-delay
TOF_Coil	timer	pt	start off-delay
TONr_Coil	timer	pt	(re)start
SP_Coil	timer	pt	start Simatic pulse
SE_Coil	timer	pt	start Simatic extended pulse
SD_Coil	timer	pt	start Simatic on-delay
SS_Coil	timer	pt	start Simatic retentive on-delay
SF_Coil	timer	pt	start Simatic of-delay
PT_Coil	timer	pt	set preset time, see ####
SC_Coil	counter	pv	set preset value
R_BitfieldCoil	operand	n	reset bits
S_BitfieldCoil	operand	n	set bits

963 Table 6 Coils with Two Operands

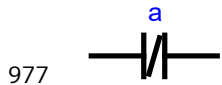
964
 965 The `SC_Coil` is another way (see 4.5.3) to set the preset value of counters. Hence the parameter names are `counter`
 966 and `pv`. The bit-field coils are a relic from MicroWin. They can be used to set or reset `n` bits in an array of `Bool` starting with
 967 the address specified by `operand`.
 968

969 4.2.3. Coils Without Operands

970 STEP 7 supports a bunch of additional contacts for AS300 and AS400 without parameter: `McrOpenCoil`,
 971 `McrCloseCoil`, `McrActivateCoil`, `McrDeactivateCoil`, and `SaveCoil`. The operand is implicit given by the
 972 instruction. Hence you must change the instruction, there is no operand.
 973

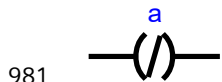
974 4.3. Negation

975 Ladder does not support negations as function block diagram does. In FBD each Boolean input or output can be negated.
 976 Instead, LAD has three instructions for the Boolean negation.



978 The `I_Contact` shows a / inside the contact, see 4.1.1. It negates the operand on top of the contact before it does its
 979 implicit Boolean `AND` with the left-side input.

980



982 The `I_Coil` shows a / inside the coil, see 4.2.1. It negates the left-side input before it assigns it to the operand on top of
 983 the coil. But, the right-side output is equal to the left-side input.

984



986 Figure 23 Not Instruction

987

988 STEP 7 also has a contact like looking `Not` which negates the left-side input and forwards the negation to the right-side
 989 output. `out := not in.`

990 The norm does not mention this. A norm like representation would be a box with a negated input or output.

991 In STEP 7 the Boolean negation is an instruction, hence its textual representation has brackets.

992 `Not ( )`

993 Listing 20 Not Instruction

994

995 Listing 20 is the textual representation of Figure 23.

996

4.4. ENO-Boxes

An ENO-box has two special pins. **EN** is a Boolean input. **ENO** is a Boolean output. The **EN**-pin has the LAD specific role rung-in. The Boolean **ENO**-pin has the LAD specific role rung-out.

While all other parameters of the instruction are simply passed through, **EN** has a generic semantic irrelevant of the instruction.

If input **EN** is false, the instruction is not executed, and **ENO** gets false. Since the instruction is not executed all other output parameters of the instruction do not influence the connected variables. If input **EN** is true, the instruction is executed, the variables connected to outputs and in-out-parameter are written and **ENO** indicates a successful execution.

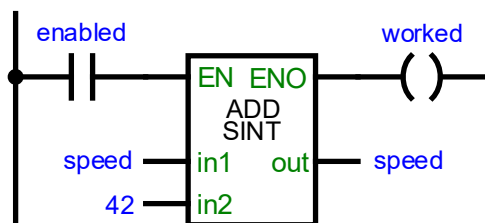


Figure 24 EN-ENO-Box

Figure 24 shows an **ADD**-box. Its **EN** input is connected to a contact, which is directly connected to the power rail. The contact reads a Boolean variable **enabled**. Its **ENO** output is connected to a coil, which writes the Boolean variable **worked**.

```
IF enabled THEN
    speed := speed + 42;
    worked := ENO;
ELSE
    worked := false;
END_IF
```

Listing 21 EN and ENO in Structured Text

In structured text the behavior of the network in Figure 24 can be expressed as shown in Listing 21. **ENO** is a system-defined temporary variable, which can be used only if a block attribute to generate **ENO** is set.

```
NETWORK
RUNG wire#powerrail
    Contact( enabled )
    {
        S7_Templates := "valueType := SInt";
        S7_GenerateENO := TRUE
    }
    Add(
        in1 := speed,
        in2 := 42,
        out := speed
    )
    Coil( worked )
END_RUNG
END_NETWORK
```

Listing 22 Generate ENO

Listing 22 shows the listing for Figure 24. Since **EN** and **ENO** are the rung pins, they are not noted in the textual representation. The box has specified **SInt** as data type for the calculation, hence the textual representation shows a pragma for a template. Since the box needs to calculate the **ENO**, the pragma **S7_GenerateENO** is **TRUE**.

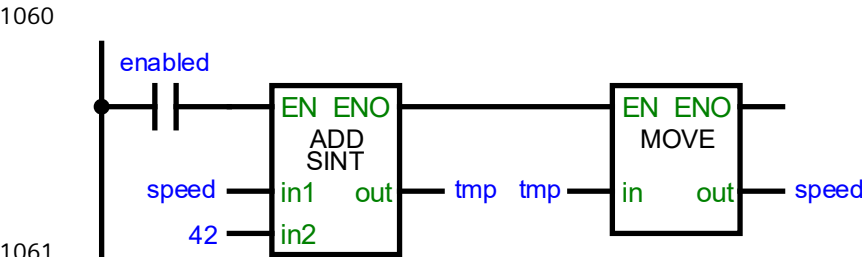
1045 If EN is false, the instruction is not executed, the variable connected to output out is not changed and ENO will be false. If
1046 enabled is true, the instruction is executed, out is in1 + in2 and worked will be changed to true in case the addition
1047 does not have a SInt overflow and to false if the sum of the inputs is greater than 127.
1048

	enabled	speed	+ 42	speed	worked
1	FALSE	-100	→	-100	FALSE
2	FALSE	100	→	100	FALSE
3	TRUE	-100	→	-58	TRUE
4	TRUE	100	→	-114	FALSE

1049 Table 7 EN-ENO Example

1050
1051 Table 7 shows four example calculations for Figure 24.
1052 Case 1 and 2: enabled is false, the instruction is not executed, speed is not changed and worked will be changed to
1053 false. Case 3 enabled is true, the instruction is executed, speed is changed to -100 + 42 and worked will be changed
1054 to true since the sum is -58. Case 4 enabled is true, the instruction is executed, speed is changed. The sum of 100 and
1055 42 is 142, but this does not fit into a SInt. The addition has an overflow, hence false is written to worked. The 8-bit
1056 overflow of 142 is 142 – 256 is -114, hence speed is changed to -114. This is not a bug, this is two-complement
1057 calculation, this is what all µ-processors are doing.
1058

1059 **4.4.1. ENO Calculation On and Off**



1061 Figure 25 Save Calculations

1062
1063
1064 Figure 25 shows what LAD can do if you are not happy with overflow results. A MOVE-box simply copies its input in to its
1065 output out. Since a MOVE-box has EN-ENO-pins, it is copying only in case if its EN is true. The result of the addition is
1066 stored in the temporary variable tmp of type SInt. If the calculation has no overflow, ENO of ADD is true and the MOVE-
1067 Box will copy it to speed. If the calculation has an overflow, the ENO of ADD is false and the MOVE will not change speed.
1068

```
1069 NETWORK
1070     RUNG wire#powerrail
1071         Contact( enabled )
1072         {
1073             S7_Templates := "valueType := SInt";
1074             S7_GenerateENO := TRUE
1075         }
1076         Add(
1077             in1 := speed,
1078             in2 := 42,
1079             out := tmp
1080         )
1081         Move(
1082             in1 := tmp,
1083             out := speed
1084         )
1085     END_RUNG
1086 END_NETWORK
```

1087 Listing 23 Save Calculation

1088

1089 Listing 23 shows the textual representation of Figure 25. Notify the two pragmas for the data type template and the ENO calculation for the Add.

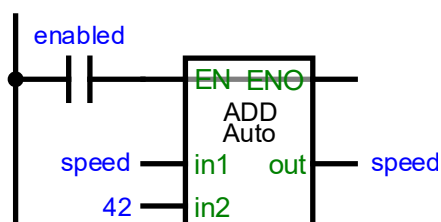
1091

1092 This easy overflow handling is not achievable with C, C++, C#. These programming languages do not offer access to the
1093 processor's flags. Python, Ruby, Java have libraries for big numbers which can handle integers with any number of bits.
1094 The processor integers must be converted before calculations, calculations are made with the help of the library, and then
1095 the big number result can be checked for the range of the target data type. This is cumbersome and slow. And often it is
1096 not necessary.

1097

1098 Although ENO generation is easily programmed, it is not easily generated. Since many programs do not need the ENO its
1099 generation is a waste of time. Therefore STEP 7 default is to switch off ENO generation for some inherent mathematical
1100 instructions. There is no way to "switch off" the semantic of the EN input.

1101



1102

1103 Figure 26 ENO Not Generated

1104 Figure 26 shows how LAD represents a switched off ENO generation. This means `ENO := EN`.

1105

```
1106 NETWORK
1107     RUNG wire#powerrail
1108         Contact( enabled )
1109         Add(
1110             in1 := speed,
1111             in2 := 42,
1112             out := speed
1113         )
1114     END_RUNG
1115 END_NETWORK
```

1116 Listing 24 ENO Not Generated

1117

1118 Listing 24 shows the textual representation for Figure 26. There are no pragmas for the addition, since `Auto` is the default
1119 for the data type template and off is the default for the ENO calculation.

1120 4.4.2. Implicit Converts

1121 Figure 26 shows `Auto` for the data type of this calculation. This means the system decides according to the data types of
1122 the input variables which data type is used for calculation. If both inputs are connected to variables or constants of the
1123 same data type, then this data type is used.

1124

1125 If different data types are connected, then the following rules are used to decide which data type the calculation uses:

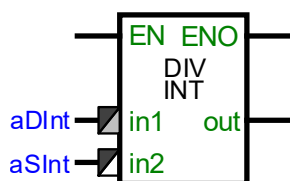
- 1126 1. float over integer – for example `Real` over `LInt`
- 1127 2. wider over smaller – for example `DInt` over `Int`
- 1128 3. signed over unsigned – for example `Int` over `UInt`
- 1129 4. ordered over unordered – for example `UInt` over `Word`
- 1130 5. Specific over unspecific – for example `Time` over `DInt`

1131 The list reflects the precedence of the rules.

1132

1133 If operands of different data types are connected or the operand has a different data type than the instruction, STEP7
 1134 offers for some instructions implicit data type conversions. These implicit conversions are graphically represented.

1135



1136

1137 Figure 27 Implicit Conversion

1138

1139 Figure 27 shows an instruction where the data type `Int` is selected. The data types of the connected variables do not fit
 1140 with the selected data type of the calculation. Hence the editor shows implicit conversions, a little diagonally split
 1141 rectangle. The first input is connected to an operand of data type `DInt`. This conversion can fail, `DInt` has a greater
 1142 range than `Int`, the implicit conversion is graphical representation shows a darker output side. The second input is
 1143 connected to an operand of data type `SInt`. This conversion cannot fail, `SInt` has a smaller range than `Int`, the implicit
 1144 conversion shows a lighter output side.

1145

1146 In case of implicit conversions, the `ENO` of the box is the logical AND of `EN`, all implicit can fail conversions, and the `ENO` of
 1147 the instruction.

1148

1149 Implicit conversions are a comfort service of STEP 7 compiler, they are not entered by the user, hence they have no
 1150 textual representation.

1151 4.4.3. Calls

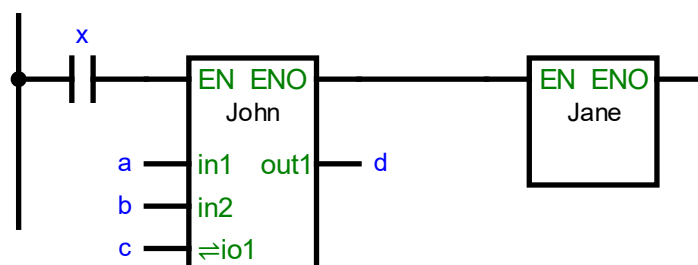
1152 Simatic PLCs offer system functions (SFC) and system function blocks (SFB). These code blocks are part of the operating
 1153 system of the PLC; they do not need to be loaded and do not require code memory.

1154 The user can define functions (FC) and function blocks (FB). The system library contains more functions (FC) and function
 1155 blocks (FB) which are prepared by TIA-Portal development. These code blocks are not part of the operating system of the
 1156 PLC; they must be loaded and require code memory.

1157 Regardless of the code block type, calls to code blocks are represented as boxes. The blocks parameter are pins of the box.
 1158 LAD implicitly adds `EN` and `ENO` pins. It is not possible to switch off `ENO` handling for calls. How the `ENO` of a code block is
 1159 specified can be found in 4.6.2.

1160

1161



1162

1163 Figure 28 Function Calls

1164

1165 Figure 28 shows two function calls. The function name, the code blocks name, is shown inside the box. `John` is a function
 1166 with two inputs, `in1` and `in2`, one in-out parameter `io1` and one output `out1`. `Jane` is a function without parameters.

1167

1168 NETWORK

```

1169  RUNG wire#powerrail
1170      Contact( x )
1171      John(
1172          in1 := a,
1173          in2 := b,
1174          io1 := c,
1175          out1 => d
1176      )
1177      Jane()
1178  END_RUNG
1179 END_NETWORK

```

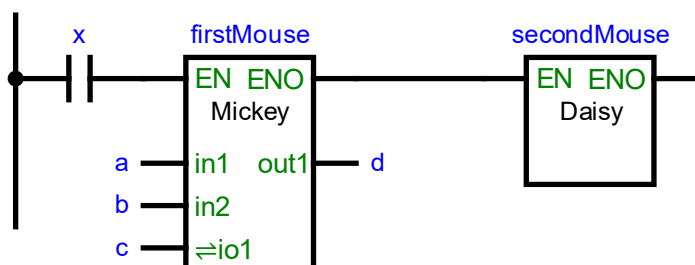
1180 Listing 25 Function Calls

1181

1182 Listing 25 shows the textual representation of Figure 28. Function calls look like inherent instructions. The function blocks
 1183 name is followed by a pair of round brackets. If the function has parameters, they are listed a line per parameter inside
 1184 the brackets.

1185

1186



1187

1188 Figure 29 Function Block Calls

1189

1190 Figure 29 shows two function block calls. The function block name, the code blocks name, is shown inside the box.

1191 Mickey is a function block with two inputs, in1 and in2, one in-out parameter io1 and one output out1. The function

1192 block instance firstMouse is displayed on top of the box. Daisy is a function block without parameters. The function

1193 block instance secondMouse is displayed on top of the box.

1194

```

1195 NETWORK
1196  RUNG wire#powerrail
1197      Contact( x )
1198      firstMouse(
1199          in1 := a,
1200          in2 := b,
1201          io1 := c,
1202          out1 => d
1203      )
1204      secondMouse()
1205  END_RUNG
1206 END_NETWORK

```

1207 Listing 26 Function Block Calls

1208

1209 Listing 26 shows the textual representation of Figure 29. Function block calls look like inherent instructions, but instead
 1210 of the function block name the instance name is given. In an error free program, the instance name is sufficient to specify
 1211 which block is called. The instance is a structured variable; its data type is the function block. A function block is a code
 1212 block as well as a data type. The instance name is followed by a pair of round brackets. If the function has parameters,
 1213 they are listed a line per parameter inside the brackets.

1214 In an incomplete program, where the user has not specified the instance, the function block name is used to identify the
 1215 block to be called. The textual representation will use the function blocks name. The network can be exported and
 1216 imported, but it cannot be compiled.

1217

4.5. Q-Boxes

There is a second set of boxes, for flip flops, timers and counters. These boxes do not have `EN` and `ENO` pins, instead they have a Boolean output named `q`. Hence the name q-boxes. The letter q is historical. It was used to avoid confusion with the digit zero which was used for the Boolean value false, and the small letter o used for negation. 0, O, and o looked too similar on the teletypewriter in the '70s.

4.5.1. Bistable Function Blocks

A flip flop is a box with two inputs, one in-out as a storing element, and one output.

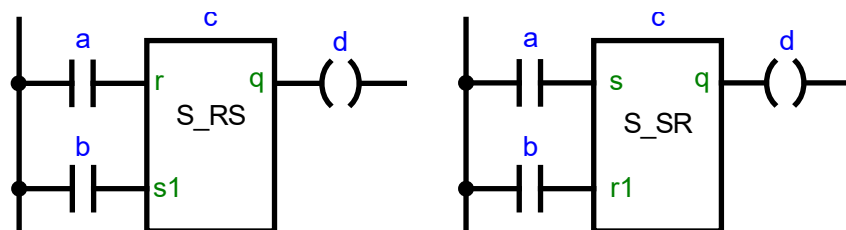


Figure 30 Simatic Flip-Flops

Figure 30 shows how already STEP 5 defined flip-flops. The second input is the dominant one. The flip-flops are inherent instructions.

1233	NETWORK	1243	NETWORK
1234	RUNG wire#powerrail	1244	RUNG wire#powerrail
1235	Contact(a)	1245	Contact(a)
1236	c.S_RS(wire#w1)	1246	c.S_SR(wire#w1)
1237	Coil(d)	1247	Coil(d)
1238	END_RUNG	1248	END_RUNG
1239	RUNG wire#powerrail	1249	RUNG wire#powerrail
1240	Contact(b)	1250	Contact(b)
1241	END_RUNG wire#w1	1251	END_RUNG wire#w1
1242	END_NETWORK	1252	END_NETWORK

Listing 27 Simatic Flip Flops

Listing 27 shows the textual representation of Figure 30. Since the first input is the rung-in, and parameter `q` is the rung-out, the dominant input remains. If the second Boolean input is connected to a variable this variable is directly listed in the brackets. But in Figure 30 the rung-out of a contact is connected to the second Boolean input. This contact is the last contact in a rung. From the rungs point of view, we now need a wire between these rungs. Hence the second rung ends with a wire. This wire is used as an operand for the second Boolean input of the box, see 3.6 and 3.7.

The storing parameter of the flip-flop, its operand, is painted on top of the box. Hence the parameter `operand` has the LAD specific role `top`. It's an in-out, treated as the instance of the box. Hence the parameter `operand` has the text specific role `instance`.

A not so nice way to represent this would be a parameter named `operand`:

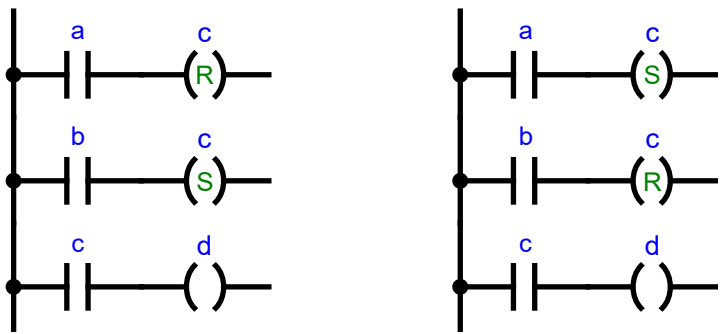
```

S_RS(
  s1 := wire#w1,
  operand := c
)
S_SR(
  r1 := wire#w1,
  operand := c
)

```

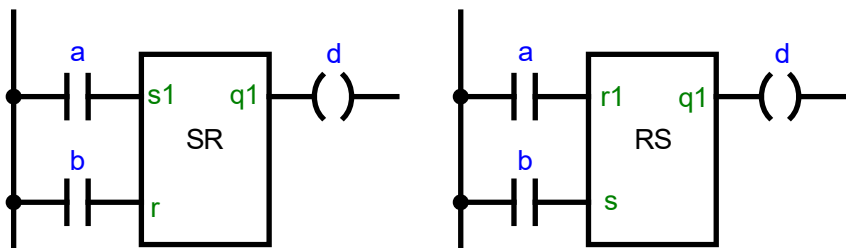
We decided against this version of the textual representation.

1274
1275



1276
1277 Figure 31 Simatic Flip Flop Equivalent

1278
1279 Figure 31 shows a three-rung equivalent for the Simatic flip-flops, using S and R Coils. This implies that the rungs are
1280 evaluated top down, which is not requested by the norm. The norm requests a top-down evaluation for networks only. A
1281 discussion about evaluation order is found in **Error! Reference source not found..**
1282
1283 The norm has a different definition of flip flops. The norm talks about bistable function blocks. But they are not defined as
1284 function blocks; they miss an instance, instead there is an output used as in-out.
1285



1286
1287 Figure 32 IEC 61131-3 Flip Flops

1288
1289 Figure 32 shows how the norm graphically represents the flip flops. The given function body is reading and writing q1.
1290 The norm's bistable function blocks are functions and parameter q1 is an in-out. The norms definition is inconsistent and
1291 confusing; STEP 7 will not support this.
1292
1293

```
1294 NETWORK
1295     RUNG wire#powerrail
1296         Contact( a )
1297         SR(
1298             r := wire#w1,
1299             // q1 := d
1300         )
1301         Coil( d )
1302     END_RUNG
1303     RUNG wire#powerrail
1304         Contact( b )
1305     END_RUNG wire#w1
1306 END_NETWORK
1307 NETWORK
1308     RUNG wire#powerrail
1309         Contact( a )
1310         RS(
1311             s := wire#w1,
1312             // q1 := d
1313         )
1314         Coil( d )
1315     END_RUNG
1316     RUNG wire#powerrail
1317         Contact( b )
1318     END_RUNG wire#w1
1319 END_NETWORK
```

Listing 28 Norm Like Flip Flops

Listing 28 shows a potential textual representation of Figure 32. The first input `s1` respective `r1` is the rung-in and parameter `q1` should be the rung-out. The second input and the in-out `q1` for the storing operand would remain. Now `q1` and `d` are used twice – very confusing. STEP 7 will not support this.

4.5.2. Timer

The norm defines three timer instructions: Pulse (TP), On-Delay (TON), Off-Delay (TOF). They all have the same interface and can be specified with different resolutions: ms (Time) or ns (LTime).

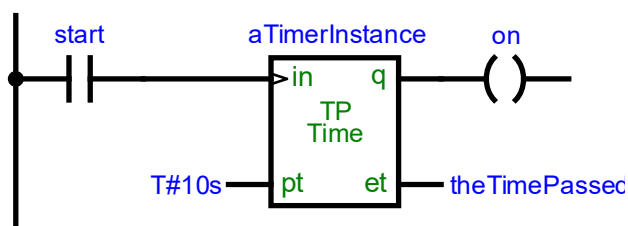


Figure 33 IEC Timer

Figure 33 shows a pulse timer. A rising edge at the input `in`, which has the role rung-in, starts the pulse. The output `q`, which has the role rung-out, will be true for the time given at input `pt`. The parameter named `timer`, has the ladder specific role `top`, hence the graphical representation shows the timers name above the box. The parameter named `timer` also has the text specific role `instance`, hence the textual representation shows it in the form `instance.method` as it is done in SCL.

This function can also be achieved by a `TP_Coil`. Timer box and timer coil use the same parameter names.

A timer box additionally has an output for the elapsed time named `et`.

```

NETWORK
  RUNG wire#powerrail
    Contact( start )
    { S7_Templates := "timeType := Time" }
    aTimerInstance.TP(
      pt := T#10s,
      et => theTimePassed
    )
    Coil( on )
  END_RUNG
END_NETWORK

```

Listing 29 IEC Timer

Listing 29 is the textual representation of Figure 33.

A `PT_Coil`, a coil with two operands, see 4.2.2, can be used to change the preset time of an already running timer. If the current elapsed time is already greater than the new preset time, the timer is immediately stopped.

A `RT_Coil`, a coil with one operand, see 4.2.1, can be used to reset all members of the timer instance.

AS1200 and AS1500 support a fourth timer, a relic from MicroWin. This timer accumulates the time its input is true. To reset this timer, you need the `RT_Coil`.

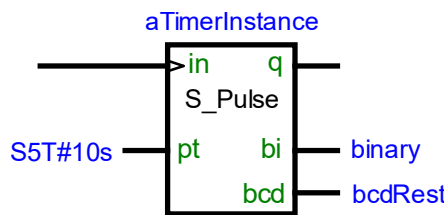


Figure 34 Simatic Timer

For compatibility reasons, there is a second set of Simatic timers. Their boxes have two outputs. The input preset time must be supplied as `S5Time`. `S5Time` is binary coded digits in 16 Bits. The high nibble is used as multiplication factor and clock selection. The remaining nibbles are used to set a counter.

Clock	Timespan
10ms	9s990ms
100ms	99s900ms
1s	16min39s
10s	166min30s

Table 8 Clock and Timespan of Simatic Timers

Table 8 shows the relation between the selected clock and the time span of the timer. Due to the encoding the maximal start value is 999. When the timer is started, the preset time is copied into its instance. The output named `q` is set according to the timer instruction, see column `q(0)` in Table 9. Every tick of the selected clock the system decrements the counter. If the counter reaches 0 the output named `q` changes its value according to the timer instruction, see column `q(pt)` in Table 9. If the input changes back before the counter reaches 0, the output changes according to column `q(in)` in Table 9.

Timer	in	q(0)	q(pt)	q(in)
<code>S_Pulse</code>	0 → 1	1	0	0
<code>S_PulseExt</code>	0 → 1	1	0	1
<code>S_OnDelay</code>	0 → 1	0	1	0
<code>S_OnDelayStay</code>	0 → 1	0	1	1
<code>S_OffDelay</code>	1 → 0	1	0	1

Table 9 Simatic Timer Instructions

The output named `bi` returns the counter value in binary encoding, so this can be directly copied into a value of type `Int`. But the value does not contain information about the selected clock, it is not directly usable as a remaining time. The output named `bcd` returns the counter value in binary coded digits and the selected clock, so this can be directly copied into a value of type `S5Time`. This value is usable as a remaining time, but due to its encoding (BCD) not directly usable in calculations. Before this is usable in calculation, it must be converted to a 32-bit `Time`.

The coils `SP_Coil`, `SE_Coil`, `SD_Coil`, `SS_Coil`, and `SF_Coil` can be used to start the Simatic timer, see Table 6. The parameter named `timer` is used as instance. The parameter named `pt` is the preset time of data type `S5Time`. The `RT_Coil` can be used to reset the timer, see Table 5. The `PT_Coil` can be used to change the preset time while a timer is running, see Table 6.

4.5.3. Counter

The norm defines three counter instructions: count up (Ctu), count down (Ctd), and count up and down (Ctud). They all can be specified to count with all integer data types.

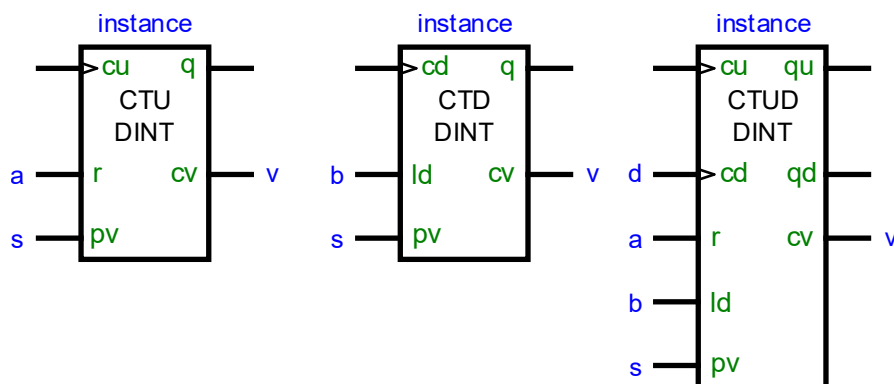


Figure 35 IEC Counter

```

{ S7_Templates := "countType := DInt" }
instance.Ctu(
    r := a,
    pv := s,
    cv => v
)
...
{ S7_Templates := "countType := DInt" }
instance.Ctd(
    ld := b,
    pv := s,
    cv => v
)
...
{ S7_Templates := "countType := DInt" }
instance.Ctud(
    cd := d,
    r := a,
    ld := b,
    pv := s,
    qd => ,
    cv => v
)

```

Listing 30 IEC Counter

Listing 30 shows the textual representations for Figure 35.

The input with name `cu` is the rung-in for `Ctu` and `Ctud`. The input with name `cd` is the rung-in for `Ctd`. The output `q` is the rung-out for `Ctu` and `Ctd`. The output `qu` is the rung-out for `Ctud`.

A rising edge on input `cu` increments the counter, a rising edge on input `cd` decrements the counter. If input `r` is true, the counter value is set to 0. If input `ld` is true, the counter value is preset to the value of input `pv`. The output named `cv` returns the current counter value. Output `qu` is `cv>=pv`. Output `qd` is `cv<=0`. Instruction `Ctu` output `q` is the same as instruction `Ctud` output `qu`. Instruction `Ctd` output `q` is the same as instruction `Ctud` output `qd`.

There are no coils for the IEC counters.

For compatibility reasons, there is a second set of counters, the legacy Simatic counters: `S_Cu`, `S_Cd`, and `S_Cud`. Their boxes have two outputs. The input preset value must be supplied by a binary coded digit. Although there is no data type `S5Count` there is a syntax for binary coded digits constants and there are conversion functions from integer data types to binary coded digits which are typically stored in unordered types. `C#100 = W#16#100 = Int#256`.

Simatic counter can use the range from C#0 to C#999. Output `cv` returns the integer value, output `cv_bcd` returns the count value as binary coded digit. The output `q` is defined as `cv > 0` for all three boxes, hence Simatic counter cannot be replaced by IEC counter.

The `CU_Coil` (see Table 5) has the same effect as the input `cu` of the box. The `CD_Coil` (see Table 5) has the same effect as the input `cd` of the box. The `SC_Coil` (see Table 6) has the same effect as the inputs named `ld` and `pv` of the box.

4.6. Jumping

Normally a μ -processor executes instruction by instruction. It has an instruction pointer to the address of the next instruction to be executed. This pointer is automatically incremented after the instruction is fetched. But there are instructions to manipulate the instruction pointer. There are four ways of setting the instruction pointer: jumps, calls, returns, and interruptions.

A **jump** changes only the instruction pointer to a given address. A **call** pushes the current instruction pointer on a stack and then it sets the instruction pointer to a given address. A **return** pops the stored address from the stack and loads it into the instruction pointer. This means that the program is continued at the address after a previously executed call. An **interrupt** is a hardware signal, which triggers a call.

All μ -processors support conditional jumping. Some support conditional instruction execution. In general, conditional execution of instructions is based on jumping. The **EN-ENO** mechanism described in 4.4 is on the one hand based on conditional execution of MC7plus instructions and on the other hand it is based on jumps and labels in MC7plus. At the end of the day conditional execution of MC7plus instructions is based on jumps and labels on assembler level.

Some programming systems do not offer interruptions; some do not offer the **EN-ENO** mechanism. But they all offer call and return, they all offer conditional jumps, sometimes hidden as control structures.

4.6.1. Call

The norm talks about program organization units (POU): functions, function blocks, and methods. In the Simatic world we talk about code blocks. A POU, a code block is the granularity for a call. A code block has a name. This name is the representation of a code address and is used as target for the jump inside the call. At the end of the called code block the execution returns to its calling code block or the operating system.

A code block prepares an environment, a chunk of memory for keeping block local data. A code block has an interface, which defines how data is stored and transported into and out of this environment.

The graphical representation of a call is just a box, see 4.4.3. The textual representation of a call depends on the code blocks type, see 4.4.3. You will find

```
Blockname()
```

or

```
Instancename()
```

In both situations the **ENO** given by the called block is used for the rung-out pin of the box.

4.6.2. Return

A return ends a code block. If the programmer does not specify the **ENO** of a block with the help of a return coil, the compiler will implicitly generate a `return true` as the last instruction executed of any code block. This guarantees that the rung-out of the box in the calling block has a defined value.

This implicit end of block execution is not the only way to return from a code block. LAD and FBD offer four return coils. If a return coil is connected to the power rail, then it represents an unconditional return, else it represents a conditional return, which returns only if the coils rung-in is true.

The four returns look like coils, three having one operand, the fourth has no operand.

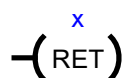
^x


Figure 36 Return Coil

The operand defines the **ENO** of the block it leaves. But these coils are four different instructions. The user must change the coil and cannot write true or false on top of it, although the graphical representation looks like.

Instruction	
<code>ReturnCoil(x)</code>	<code>Eno := x</code>
<code>ReturnFalse()</code>	<code>Eno := false</code>
<code>ReturnTrue()</code>	<code>Eno := true</code>
<code>Return()</code>	<code>Eno := rung-in</code>

Table 10 Return Instructions

The `ReturnCoil()` has one operand. It must be connected to a Boolean variable. The variable's value is returned in the ENO as the rung-out of the calling box. `ReturnFalse()` returns false in the ENO as the rung-out of the calling box.

`ReturnTrue()` returns true in the ENO as the rung-out of the calling box.

`Return()` returns the rung-in of the coil in the ENO as the rung-out of the calling box. In case of LAD this is the same as a `ReturnTrue()`, since the instruction only returns if rung-in is true. In AWL an unconditional return returning the RLO makes sense.

Return coils do not have a pin marked as rung-out – there is no right side where something can be connected to, since if they return, there is nothing else evaluated in this block. Since a return coil has no rung-out it must always be the last instruction in a rung, see 4.6.5.

4.6.3. Labels

Calls are jumps into a block, to the first instruction of a block. The block's name is used to identify the code address necessary for the jump. The call stores the return address, the instruction following the call, on a stack.

It is the same for interruptions, which are calls from the operating system to an organization block. The interrupt stores the code address of the interrupted instruction as return address on the stack. A return-coil jumps out of organization block and back to the interrupted block.

Labels are used to identify instruction addresses inside a code block. Labels have a name; the name must be unique inside a code block. Labels use a separate scope; a label name can be identical to a local or global symbol.

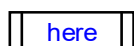


Figure 37 A Label

The graphical representation of a label is a special box without a rung-in or rung-out. They are not connected to the powerline. This box addresses a network. There is no way to address an instruction inside the network. The reason is described in **Error! Reference source not found..**

```
{ S7_Language := "LAD" }
NETWORK
  Label( here )
  RUNG wire#powerrail
```

Listing 31 A Label

The textual representation looks like an instruction, with the label as operand. Since a label instruction can only have a label as operand, there is no need to indicate the scope of the label. There is no need to write it as `label#here` or as it necessary in SCL as `label:`.

The label instruction follows directly in the line after the keyword `NETWORK`. It is not allowed to define a label anywhere else, for example inside a rung. This guarantees the semantic of a network.

Today it is not allowed to have several labels for a network. Sometimes, when a block is revised, this would be handy but can be achieved by an empty network.

4.6.4. Jumps

A jump is a coil with a label as operand. If a jump coil is connected to the power rail, then it represents an unconditional jump, else it represents a conditional jump. Since the label addresses a network, the jump will stop execution of the network it is in and continue with the execution of the network addressed by the label.

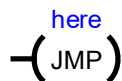


Figure 38 A Jump Coil

Two different jump coils are defined:

Instruction	
JumpCoil(...)	jump if rung-in true
I_JumpCoil(...)	jump if rung-in false

Table 11 Jump Coils

```
JumpCoil( here )
```

Listing 32 A Jump Coil

Listing 32 shows the textual representation of Figure 38.

The `JumpCoil()` has one operand – a label. Since a jump instruction can only have a label as operand, there is no need to indicate the scope of the label. There is no need to write it as `label#here`.

Jump coils do not have a pin marked as rung-out – there is no right side where something can be connected to, since there is nothing else evaluated in this network if they jump. Since a jump coil has no rung-out it must always be the last instruction in a rung, see 4.6.5.

4.6.5. Only One Jump or Return

In textual languages the statements are executed one by one, the evaluation order follows the text. What is listed first, is done first. AWL offers labels and jumps, and no one discusses what is executed first. But no one likes spaghetti-code.

Hence structured text has instructions to control the execution of the instructions: `IF-THEN-ELSE`, `WHILE ... DO`, `REPEAT ... UNTIL`. The semantic of these instructions achieves a clear evaluation order.

Graphical programming languages do not define an evaluation order, see **Error! Reference source not found.** This is no problem until the block has no feedback (see **Error! Reference source not found.**) and only one jump or return is found. Several jumps or returns in a block raises the question, which one is evaluated so to say executed first. Due to lack of clear advice by the norm, see **Error! Reference source not found.**, TIA-Portal must make its own rules:

1. If a block has more than one network, the networks are evaluated top to bottom.
2. There must be only one jump or return coil inside a network.

The first rule allows you to create the same spaghetti code as you can do in AWL. You just split the basic blocks, instruction sequences without jumps and labels, into separate networks. The second rule avoids ambiguities in respect to the evaluation order. Chapter **Error! Reference source not found.** will discuss more aspects of this, but for here it is sufficient to consider two of the possible evaluation orders: column first versus row first.

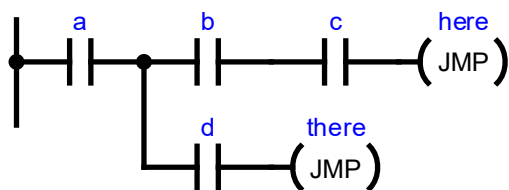


Figure 39 A Net with Two Jumps

Figure 39 shows a network with two jumps. Assume all variables are true, so both jumps would jump, since their rung-ins gets true. But the μ -processor executing this assembler will not evaluate all instructions in parallel and execute both jumps. It can only execute one assembler instruction after the other, so it will jump only once. If the instruction pointer is set to *here*, the remaining assembler of the network is not executed. This means no jumping *there*. If the instruction pointer is set to *there*, ...

The semantic of a network is a matter of its evaluation order.

Evaluation order **top-down-left-right** means that the graph is evaluated inside the column from top to bottom and column by column, from left to right. This is driven by the idea that an instruction can only be evaluated if all inputs are given. Since ladder emulates the flow of electric power, going column by column from left to right, it is kind of natural and should always find all inputs evaluated. Inside the column evaluation can be defined as top-down or bottom-up.

For a top-down-left-right evaluation order the "assembler" of Figure 39 could look like Listing 33.

```
Load true
And a
Store roa // rung-out of contact(a)
Load roa
And b
Store rob // rung-out of contact(b)
Load roa
And d
Store rod // rung-out of contact(d)
Load roa
And c
Store roc // rung-out of contact(c)
Load rod
Jmp there
Load roa
Jmp here
```

Listing 33 Top-Down-Left-Right Assembler

This is a very simple, instruction by instruction compilation, an easy to implement compiler, with low memory footprint for the compiler, perfect for small systems. If the compiler generates the code according to column first the network would jump to *there*. In the case of column wise code generation, the leftmost jump in a network wins.

Evaluation order left-right-top-down means that the graph is evaluated row by row, from top to bottom. Inside a row, evaluation needs to go from left to right, otherwise it misses an evaluated rung-in.

For a left-right-top-down evaluation order the "assembler" of Figure 39 could look like Listing 34.

```
Load true
And a
Store w1 // rung-out of contact(a) and input for wire#w1
And b
And c
Jmp here
Load w1
And d
Jmp there
```

Listing 34 Left-Right-Top-Down Assembler

If the compiler generates the code according to left-right-top-down evaluation order the network would jump to [here](#). In case of a row wise code generation the topmost jump in a network wins.

Today, due to our history, the TIA-Portal compiler for LAD and FBD do neither column- nor row-wise code generation, see chapter **Error! Reference source not found..** These rules limit optimization. To enable the best optimization TIA-Portal limits the number of jumps/returns inside a network. There can be only one. The jump is going to generate the last assembler instruction of this network.

Allowing only one jump or return coil avoids ambiguities and allows optimized code generation.

Allowing only one jump in a network, leads to a lot of small networks in situations where SCL programmers can use a `switch-case` control structure. Therefore, LAD introduced two instructions: `JumpList` and `Switch`.

4.6.6. Jump List

A jump list is a calculated jump. One integer input decides which label the instruction jumps to. This is the basis of a `switch-case` in SCL. But in SCL the cases can be lists of values, even ranges. On the assembler side, one address calculation is sufficient to determine the target. This is very fast.

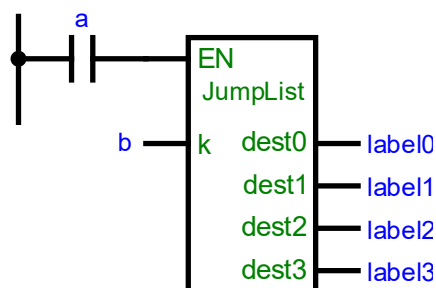


Figure 40 Jump List Box

A jump list looks like a box. It has `EN` but no `ENO`, since it acts like a jump. If `EN` is false, the box does not jump, and the program execution continues with the next network. If `EN` is true input `k` selects the label the box jumps to. If `k` is 0 then the label at `dest0` is used, if `k` is 1 then the label at `dest1` is used, if `k` is 2 then the label at `dest2` is used, and so on. If `k` is greater than the last listed `dest`, then the box does not jump, so program execution continues with the next network in the block.

The box shows the destination labels on the right side as if they are outputs. But destination parameters are no outputs, there is no data flowing into the label.

```
{ S7_Language := "LAD" }
NETWORK
  RUNG wire#powerrail
    Contact( a )
    JumpList(
      k := b,
      dest0 => label0,
      dest1 => label1,
      dest2 => label2,
      dest3 => label3,
    )
  END_RUNG
END_NETWORK
```

Listing 35 Jump List

Listing 35 shows the textual representation of the jump list box of Figure 40. Since the box shows the destination label on the right side as if they are outputs, the textual representation follows this and uses the syntax of outputs. Although they

are no outputs. In respect to the information flow, they are inputs, since they hand over the instruction the information where it shall jump to.

The jump list does not have a rung-out. It is a jump. Hence it is always the last instruction in a rung.

4.6.7. Switch

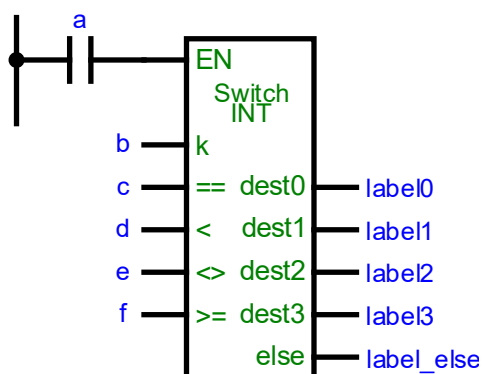


Figure 41 Switch Box

A switch looks like a box. It has **EN** but no **ENO**, since it acts as a jump. If **EN** is false, the box does not jump, and the program execution continues with the next network. The user must select a data type. All inputs must be connected to variables or constants of that data type. If **EN** is true, the box evaluates a list of comparisons. The user can select for each input if the relation shall be **==**, **<>**, **<**, **>**, **<=**, **>=**. The first comparison evaluating true is selecting the label jumped to. If no comparison returns true, then the "else-label" is selected.

```
{ S7_Language := "LAD" }
NETWORK
  RUNG wire#powerrail
    Contact( a )
    Switch(
      k := b,
      EQ c => label0,
      LT d => label1,
      NE e => label2,
      GE f => label3,
      else => label_else
    )
  END_RUNG
END_NETWORK
```

Listing 36 Switch

Listing 36 shows the textual representation of Figure 41. The interesting thing is the handling of the relations. The relations are treated like keywords. Each line represents a case. For example, **LT d => label1** expresses jump to **label1** if **b < d**. The general form is

```
relation symbol => label
```

which is close to the treating of negated Boolean output parameters in FBD. This one-line representation avoids any confusing counting of relation number, input number and destination number. One line instead of three.

```
if a then
  if b == c
  then goto label0
  else if b < d
    then goto label1
    else if b <> e
      then goto label2
      else if b >= f
        then goto label3
        else goto label_else
      end_if
    end_if
  end_if
end_if
```

Listing 37 Switch Box Equivalent In SCL

Listing 37 shows a SCL equivalent for Figure 41. The switch box is more flexible than the switch instruction of SCL. The switch-case of SCL is closer to a jump list box, than to a switch box.

4.7. Calculating

LAD is a preferred language for Boolean expressions, and it offers all kinds of arithmetic boxes. But implementing a non-Boolean expression is cumbersome, since you cannot wire non-Boolean pins. A non-Boolean pin can be connected to variable or constants only. Writing a simple expression like

```
c := sqr( sin( a )) + sqr( cos( b ))
```

in LAD needs four temporary variables additionally to the five boxes. To avoid these temporary variables LAD offers a box named **Calculate**.

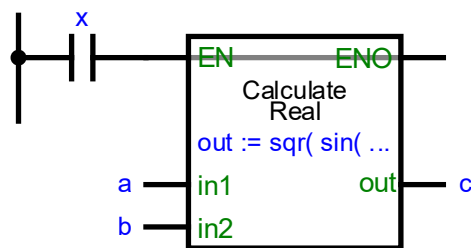


Figure 42 Calculate Box

The calculate box is extensible, the user can add inputs. The user must select a data type. This is the data type for the calculation and defines the set of usable operations. Variables of different data types are implicitly converted, which is indicated according to 4.4.2. In the middle of the box, the user can enter an expression using available operations for the data type.

```
RUNG wire#powerrail
  Contact( x )
  {
    S7_Templates := "SrcType := Real";
    S7_Expression := "sqr( sin( in1 )) + sqr( cos( in2 ))"
  }
  Calculate(
    in1 := a,
    in2 := b,
    out => c
  )
END_RUNG
```

Listing 38 Calculate Box

Listing 38 shows the textual representation of Figure 42. The interesting point is the handling of the expression. The pragma `S7_Expression` is used to store it. As in the editor the expression uses the formal names of the parameters: `in1`, `in2`, and so on. There is no `out :=` since this is not changeable in any way.

5. Export, Import

So far, mostly good situations are described -- compile clean networks. Even if the networks are not compile-clean, maybe you are in the middle of a greater change, the textual representation must be able to express what the STEP 7 editor shows.

When the textual representation is imported STEP 7 can run into different situations. The parser finds an identifier followed by an open round bracket. It will first search in the inherent instructions, then in the local variables, then in system library and finally in the user program for all identifiers in the global namespace. Depending on if and where the identifier is found STEP 7 will prepare.

- Instruction → contact, coil, or box
- Local variable → box
 - if instance of SFB or FB, with the data types name inside the box and the instance on top of the box
 - else, with the instance on top of the red box
- SFC or FC → box, with the blocks name inside the box
- SFB or FB → box, with the blocks name inside the red box, the editor will show three red question marks on top of the box, to indicate the missing instance
- Instance → box with the data types name inside the box and the instance on top of the box
- else → red box, with the identifier inside the box

If no closing round bracket is found, the network is rejected and the import is interrupted with a syntax error. The next step is to compare the list of parameters. If the imported parameters do not fit to the found object, the box will be shown red with the imported list of parameters, but the import continues.

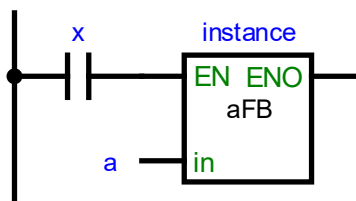


Figure 43 Instance Type Fits to Block Type

Figure 43 shows the graphical representation if a variable named `instance` is of data type `aFB` and the list of parameter fits to the interface of a function block named `aFB`.

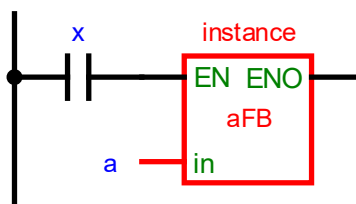


Figure 44 Instance Type does not Fit to Blocks Interface

Figure 44 shows the graphical representation if a variable named `instance` is of data type `aFB` but the list of parameters does not fit to the interface of a function block named `aFB`.

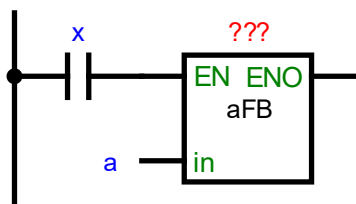


Figure 45 Instance Missing

Figure 45 shows the graphical representation if a function block named `aFB` is found and the list of parameter fits to the interface of that function block.

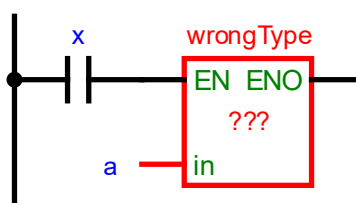


Figure 46 Unknown Instance Data Type

Figure 46 shows the graphical representation if the data type of a variable named `instance` is unknown. There is no function block with the name of the data type, or the data types name is something else but a function blocks name. The list of parameters is shown.

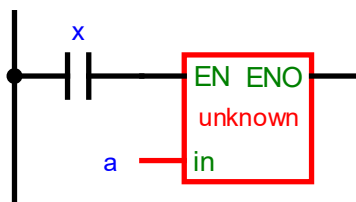


Figure 47 Unknown Identifier

Figure 47 shows the graphical representation if the identifier in the textual representation is not found, or the found object is not a function or function block. The list of parameters is shown.

6. Appendix

6.1. Service and support

SiePortal

The integrated platform for product selection, purchasing and support - and connection of Industry Mall and Online support. The SiePortal home page replaces the previous home pages of the Industry Mall and the Online Support Portal (SIOS) and combines them.

- **Products & Services**
In Products & Services, you can find all our offerings as previously available in Mall Catalog.
- **Support**
In Support, you can find all information helpful for resolving technical issues with our products.
- **mySieportal**
mySiePortal collects all your personal data and processes, from your account to current orders, service requests and more. You can only see the full range of functions here after you have logged in.

You can access SiePortal via this address: sieportal.siemens.com

Technical Support

The Technical Support of Siemens Industry provides you fast and competent support regarding all technical queries with numerous tailor-made offers – ranging from basic support to individual support contracts.

Please send queries to Technical Support via Web form: support.industry.siemens.com/cs/my/src

SITRAIN – Digital Industry Academy

We support you with our globally available training courses for industry with practical experience, innovative learning methods and a concept that's tailored to the customer's specific needs.

For more information on our offered trainings and courses, as well as their locations and dates, refer to our web page: siemens.com/sitrain

Industry Online Support app

You will receive optimum support wherever you are with the "Industry Online Support" app. The app is available for iOS and Android:



6.2. Links and literature

No.	Topic
\1\	Siemens Industry Online Support https://support.industry.siemens.com
\2\	Link to this entry page of this application example https://support.industry.siemens.com/cs/ww/en/view/109994073
\3\	

6.3. Change documentation

Version	Date	Modification
V1.0	09/2025	First version